iir1

Generated on Mon Oct 19 2026 18:26:30 for iir1 by Doxygen 1.15.0

Mon Oct 19 2026 18:26:30

1 DSP IIR Realtime C++ filter library 1

2 Namespace Index 1

2.1 Namespace List . . . 1

3 Hierarchical Index 1

3.1 Class Hierarchy . . . 1

4 Class Index 5

4.1 Class List . . . 5

5 File Index 8

5.1 File List . . . 8

6 Namespace Documentation 8

6.1 Iir Namespace Reference . . . 8

6.1.1 Detailed Description . . . 9

6.2 Iir::Butterworth Namespace Reference . . . 11

6.2.1 Detailed Description . . . 11

6.3 Iir::ChebyshevI Namespace Reference . . . 11

6.3.1 Detailed Description . . . 11

6.4 Iir::ChebyshevII Namespace Reference . . . 12

6.4.1 Detailed Description . . . 12

6.5 Iir::Custom Namespace Reference . . . 12

6.5.1 Detailed Description . . . 12

7 Class Documentation 12

7.1 Iir::RBJ::AllPass Struct Reference . . . 12

7.1.1 Detailed Description . . . 13

7.1.2 Member Function Documentation . . . 13

7.2 Iir::Butterworth::AnalogLowPass Class Reference . . . 14

7.2.1 Detailed Description . . . 14

7.3 Iir::ChebyshevI::AnalogLowPass Class Reference . . . 14

7.3.1 Detailed Description . . . 14

7.4 Iir::ChebyshevII::AnalogLowPass Class Reference . . . 14

7.4.1 Detailed Description . . . 15

7.5 Iir::Butterworth::AnalogLowShelf Class Reference . . . 15

7.5.1 Detailed Description . . . 15

7.6 Iir::ChebyshevI::AnalogLowShelf Class Reference . . . 15

7.6.1 Detailed Description . . . 15

7.7 Iir::ChebyshevII::AnalogLowShelf Class Reference . . . 15

7.7.1 Detailed Description . . . 16

7.8 Iir::Butterworth::BandPass< FilterOrder, StateType > Struct Template Reference . . . 16

7.8.1 Detailed Description . . . 16

7.8.2 Member Function Documentation . . . 16
7.9 Iir::ChebyshevI::BandPass< FilterOrder, StateType > Struct Template Reference . . . 18
7.9.1 Detailed Description . . . 18
7.9.2 Member Function Documentation . . . 18
7.10 Iir::ChebyshevII::BandPass< FilterOrder, StateType > Struct Template Reference . . . 20
7.10.1 Detailed Description . . . 20
7.10.2 Member Function Documentation . . . 20
7.11 Iir::RBJ::BandPass1 Struct Reference . . . 22
7.11.1 Detailed Description . . . 23
7.11.2 Member Function Documentation . . . 23
7.12 Iir::RBJ::BandPass2 Struct Reference . . . 23
7.12.1 Detailed Description . . . 24
7.12.2 Member Function Documentation . . . 24
7.13 Iir::Butterworth::BandPassBase Struct Reference . . . 25
7.14 Iir::ChebyshevI::BandPassBase Struct Reference . . . 25
7.15 Iir::ChebyshevII::BandPassBase Struct Reference . . . 26
7.16 Iir::BandPassTransform Class Reference . . . 26
7.16.1 Detailed Description . . . 27
7.17 Iir::Butterworth::BandShelf< FilterOrder, StateType > Struct Template Reference . . . 27
7.17.1 Detailed Description . . . 27
7.17.2 Member Function Documentation . . . 27
7.18 Iir::ChebyshevI::BandShelf< FilterOrder, StateType > Struct Template Reference . . . 29
7.18.1 Detailed Description . . . 29
7.18.2 Member Function Documentation . . . 30
7.19 Iir::ChebyshevII::BandShelf< FilterOrder, StateType > Struct Template Reference . . . 31
7.19.1 Detailed Description . . . 32
7.19.2 Member Function Documentation . . . 32
7.20 Iir::RBJ::BandShelf Struct Reference . . . 33
7.20.1 Detailed Description . . . 34
7.20.2 Member Function Documentation . . . 34
7.21 Iir::Butterworth::BandShelfBase Struct Reference . . . 35
7.22 Iir::ChebyshevI::BandShelfBase Struct Reference . . . 36
7.23 Iir::ChebyshevII::BandShelfBase Struct Reference . . . 36
7.24 Iir::Butterworth::BandStop< FilterOrder, StateType > Struct Template Reference . . . 37
7.24.1 Detailed Description . . . 37
7.24.2 Member Function Documentation . . . 37
7.25 Iir::ChebyshevI::BandStop< FilterOrder, StateType > Struct Template Reference . . . 38
7.25.1 Detailed Description . . . 39
7.25.2 Member Function Documentation . . . 39
7.26 Iir::ChebyshevII::BandStop< FilterOrder, StateType > Struct Template Reference . . . 41
7.26.1 Detailed Description . . . 41
7.26.2 Member Function Documentation . . . 41

7.27 Iir::RBJ::BandStop Struct Reference . . . 43
7.27.1 Detailed Description . . . 44
7.27.2 Member Function Documentation . . . 44
7.28 Iir::Butterworth::BandStopBase Struct Reference . . . 44
7.29 Iir::ChebyshevI::BandStopBase Struct Reference . . . 45
7.30 Iir::ChebyshevII::BandStopBase Struct Reference . . . 45
7.31 Iir::BandStopTransform Class Reference . . . 46
7.31.1 Detailed Description . . . 46
7.32 Iir::Biquad Class Reference . . . 46
7.32.1 Member Function Documentation . . . 47
7.33 Iir::BiquadPoleState Struct Reference . . . 49
7.33.1 Detailed Description . . . 49
7.34 Iir::Cascade Class Reference . . . 49
7.34.1 Detailed Description . . . 50
7.34.2 Member Function Documentation . . . 50
7.35 Iir::CascadeStages< MaxStages, StateType > Class Template Reference . . . 51
7.35.1 Detailed Description . . . 51
7.35.2 Member Function Documentation . . . 51
7.36 Iir::ComplexPair Struct Reference . . . 52
7.36.1 Detailed Description . . . 52
7.36.2 Member Function Documentation . . . 52
7.37 Iir::DirectFormI Class Reference . . . 52
7.37.1 Detailed Description . . . 52
7.38 Iir::DirectFormII Class Reference . . . 53
7.38.1 Detailed Description . . . 53
7.39 Iir::Butterworth::HighPass< FilterOrder, StateType > Struct Template Reference . . . 53
7.39.1 Detailed Description . . . 53
7.39.2 Member Function Documentation . . . 54
7.40 Iir::ChebyshevI::HighPass< FilterOrder, StateType > Struct Template Reference . . . 55
7.40.1 Detailed Description . . . 55
7.40.2 Member Function Documentation . . . 55
7.41 Iir::ChebyshevII::HighPass< FilterOrder, StateType > Struct Template Reference . . . 57
7.41.1 Detailed Description . . . 57
7.41.2 Member Function Documentation . . . 57
7.42 Iir::RBJ::HighPass Struct Reference . . . 59
7.42.1 Detailed Description . . . 59
7.42.2 Member Function Documentation . . . 60
7.43 Iir::Butterworth::HighPassBase Struct Reference . . . 60
7.44 Iir::ChebyshevI::HighPassBase Struct Reference . . . 61
7.45 Iir::ChebyshevII::HighPassBase Struct Reference . . . 61
7.46 Iir::HighPassTransform Class Reference . . . 62
7.46.1 Detailed Description . . . 62

7.47 Iir::Butterworth::HighShelf< FilterOrder, StateType > Struct Template Reference 62
7.47.1 Detailed Description 62
7.47.2 Member Function Documentation 62
7.48 Iir::ChebyshevI::HighShelf< FilterOrder, StateType > Struct Template Reference 64
7.48.1 Detailed Description 64
7.48.2 Member Function Documentation 64
7.49 Iir::ChebyshevII::HighShelf< FilterOrder, StateType > Struct Template Reference 66
7.49.1 Detailed Description 66
7.49.2 Member Function Documentation 67
7.50 Iir::RBJ::HighShelf Struct Reference 68
7.50.1 Detailed Description 69
7.50.2 Member Function Documentation 69
7.51 Iir::Butterworth::HighShelfBase Struct Reference 70
7.52 Iir::ChebyshevI::HighShelfBase Struct Reference 70
7.53 Iir::ChebyshevII::HighShelfBase Struct Reference 71
7.54 Iir::RBJ::IIRNotch Struct Reference 71
7.54.1 Detailed Description 72
7.54.2 Member Function Documentation 72
7.55 Iir::Layout< MaxPoles > Class Template Reference 73
7.55.1 Detailed Description 73
7.56 Iir::LayoutBase Class Reference 73
7.56.1 Detailed Description 73
7.57 Iir::Butterworth::LowPass< FilterOrder, StateType > Struct Template Reference 73
7.57.1 Detailed Description 74
7.57.2 Member Function Documentation 74
7.58 Iir::ChebyshevI::LowPass< FilterOrder, StateType > Struct Template Reference 75
7.58.1 Detailed Description 75
7.58.2 Member Function Documentation 76
7.59 Iir::ChebyshevII::LowPass< FilterOrder, StateType > Struct Template Reference 77
7.59.1 Detailed Description 77
7.59.2 Member Function Documentation 78
7.60 Iir::RBJ::LowPass Struct Reference 79
7.60.1 Detailed Description 80
7.60.2 Member Function Documentation 80
7.61 Iir::Butterworth::LowPassBase Struct Reference 80
7.62 Iir::ChebyshevI::LowPassBase Struct Reference 81
7.63 Iir::ChebyshevII::LowPassBase Struct Reference 81
7.64 Iir::LowPassTransform Class Reference 82
7.64.1 Detailed Description 82
7.65 Iir::Butterworth::LowShelf< FilterOrder, StateType > Struct Template Reference 82
7.65.1 Detailed Description 83
7.65.2 Member Function Documentation 83

7.66 Iir::ChebyshevI::LowShelf< FilterOrder, StateType > Struct Template Reference 84
7.66.1 Detailed Description 85
7.66.2 Member Function Documentation 85
7.67 Iir::ChebyshevII::LowShelf< FilterOrder, StateType > Struct Template Reference 86
7.67.1 Detailed Description 87
7.67.2 Member Function Documentation 87
7.68 Iir::RBJ::LowShelf Struct Reference 88
7.68.1 Detailed Description 89
7.68.2 Member Function Documentation 89
7.69 Iir::Butterworth::LowShelfBase Struct Reference 90
7.70 Iir::ChebyshevI::LowShelfBase Struct Reference 90
7.71 Iir::ChebyshevII::LowShelfBase Struct Reference 91
7.72 Iir::Custom::OnePole Struct Reference 91
7.72.1 Detailed Description 92
7.73 Iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles > Struct Template Reference 92
7.73.1 Detailed Description 93
7.74 Iir::PoleFilterBase< AnalogPrototype > Class Template Reference 93
7.74.1 Detailed Description 93
7.75 Iir::PoleFilterBase2 Class Reference 93
7.75.1 Detailed Description 94
7.76 Iir::PoleZeroPair Struct Reference 94
7.76.1 Detailed Description 94
7.77 Iir::RBJ::RBJbase Struct Reference 95
7.77.1 Detailed Description 96
7.78 Iir::Custom::SOSCascade< NSOS, StateType > Struct Template Reference 96
7.78.1 Detailed Description 96
7.78.2 Constructor & Destructor Documentation 97
7.78.3 Member Function Documentation 97
7.79 Iir::Cascade::Storage Struct Reference 98
7.79.1 Detailed Description 98
7.80 Iir::TransposedDirectFormII Class Reference 98
7.81 Iir::Custom::TwoPole Struct Reference 98
7.81.1 Detailed Description 99

8 File Documentation **99**

8.1 Iir.h 99
8.2 Biquad.h 99
8.3 Butterworth.h 100
8.4 Cascade.h 105
8.5 ChebyshevI.h 106
8.6 ChebyshevII.h 112
8.7 Common.h 117

8.8 Custom.h . . . 118
8.9 Layout.h . . . 118
8.10 MathSupplement.h . . . 120
8.11 PoleFilter.h . . . 121
8.12 RBJ.h . . . 123
8.13 State.h . . . 125
8.14 Types.h . . . 126

Index **129**

1 DSP IIR Realtime C++ filter library

An infinite impulse response (IIR) filter library for Linux, Mac OSX and Windows which implements Butterworth, RBJ, Chebychev filters and can easily import coefficients generated by Python (scipy).

The filter processes the data sample by sample for realtime processing.

It uses templates to allocate the required memory so that it can run without any malloc / new commands. Memory is allocated at compile time so that there is never any risk of memory leaks.

This library has been further developed from Vinnie Falco's great original work which can be found here:

https://github.com/vinniefalco/DSPFilters

Bernd Porr – http://www.berndporr.me.uk

2 Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

Iir **8**

Iir::Butterworth **11**

Iir::ChebyshevI **11**

Iir::ChebyshevII **12**

Iir::Custom **12**

3 Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BandPassBase

Iir::PoleFilter< BandPassBase, DirectFormII, 4, 4 *2 > 92

Iir::Butterworth::BandPass< FilterOrder, StateType > 16

Iir::ChebyshevI::BandPass< FilterOrder, StateType > 18

Iir::ChebyshevII::BandPass< FilterOrder, StateType > 20

Iir::BandPassTransform 26
BandShelfBase

Iir::PoleFilter< BandShelfBase, DirectFormII, 4, 4 *2 > 92

Iir::Butterworth::BandShelf< FilterOrder, StateType > 27

Iir::ChebyshevI::BandShelf< FilterOrder, StateType > 29

Iir::ChebyshevII::BandShelf< FilterOrder, StateType > 31
BandStopBase

Iir::PoleFilter< BandStopBase, DirectFormII, 4, 4 *2 > 92

Iir::Butterworth::BandStop< FilterOrder, StateType > 37

Iir::ChebyshevI::BandStop< FilterOrder, StateType > 38

Iir::ChebyshevII::BandStop< FilterOrder, StateType > 41

Iir::BandStopTransform 46
BaseClass

Iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles > 92

Iir::Biquad 46

Iir::Custom::OnePole 91

Iir::Custom::TwoPole 98

Iir::RBJ::RBJbase 95

Iir::RBJ::AllPass 12

Iir::RBJ::BandPass1 22

Iir::RBJ::BandPass2 23

Iir::RBJ::BandShelf 33

Iir::RBJ::BandStop 43

Iir::RBJ::HighPass 59

Iir::RBJ::HighShelf 68

Iir::RBJ::IIRNotch 71

Iir::RBJ::LowPass 79

Iir::RBJ::LowShelf 88

Iir::Cascade 49

- iir::PoleFilterBase2 93
  - iir::PoleFilterBase< AnalogLowPass > 93
    - iir::Butterworth::BandPassBase 25
    - iir::Butterworth::BandStopBase 44
    - iir::Butterworth::HighPassBase 60
    - iir::Butterworth::LowPassBase 80
    - iir::ChebyshevI::BandPassBase 25
    - iir::ChebyshevI::BandStopBase 45
    - iir::ChebyshevI::HighPassBase 61
    - iir::ChebyshevI::LowPassBase 81
    - iir::ChebyshevII::BandPassBase 26
    - iir::ChebyshevII::BandStopBase 45
    - iir::ChebyshevII::HighPassBase 61
    - iir::ChebyshevII::LowPassBase 81
  - iir::PoleFilterBase< AnalogLowShelf > 93
    - iir::Butterworth::BandShelfBase 35
    - iir::Butterworth::HighShelfBase 70
    - iir::Butterworth::LowShelfBase 90
    - iir::ChebyshevI::BandShelfBase 36
    - iir::ChebyshevI::HighShelfBase 70
    - iir::ChebyshevI::LowShelfBase 90
    - iir::ChebyshevII::BandShelfBase 36
    - iir::ChebyshevII::HighShelfBase 71
    - iir::ChebyshevII::LowShelfBase 91
  - iir::PoleFilterBase< AnalogPrototype > 93
- iir::CascadeStages< MaxStages, StateType > 51
- iir::CascadeStages< NSOS, DirectFormII > 51
  - iir::Custom::SOSCascade< NSOS, StateType > 96
- iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII > 51
  - iir::PoleFilter< HighPassBase, DirectFormII, 4 > 92
    - iir::Butterworth::HighPass< FilterOrder, StateType > 53
    - iir::ChebyshevI::HighPass< FilterOrder, StateType > 55

Iir::ChebyshevII::HighPass< FilterOrder, StateType > 57

Iir::PoleFilter< HighShelfBase, DirectFormII, 4 > 92

Iir::Butterworth::HighShelf< FilterOrder, StateType > 62

Iir::ChebyshevI::HighShelf< FilterOrder, StateType > 64

Iir::ChebyshevII::HighShelf< FilterOrder, StateType > 66

Iir::PoleFilter< LowPassBase, DirectFormII, 4 > 92

Iir::Butterworth::LowPass< FilterOrder, StateType > 73

Iir::ChebyshevI::LowPass< FilterOrder, StateType > 75

Iir::ChebyshevII::LowPass< FilterOrder, StateType > 77

Iir::PoleFilter< LowShelfBase, DirectFormII, 4 > 92

Iir::Butterworth::LowShelf< FilterOrder, StateType > 82

Iir::ChebyshevI::LowShelf< FilterOrder, StateType > 84

Iir::ChebyshevII::LowShelf< FilterOrder, StateType > 86

Iir::CascadeStages<(MaxAnalogPoles+1)/2, StateType > 51

Iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles > 92

Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII > 51

Iir::PoleFilter< BandPassBase, DirectFormII, 4, 4 ∗2 > 92

Iir::PoleFilter< BandShelfBase, DirectFormII, 4, 4 ∗2 > 92

Iir::PoleFilter< BandStopBase, DirectFormII, 4, 4 ∗2 > 92

complex_pair_t

Iir::ComplexPair 52

Iir::DirectFormI 52

Iir::DirectFormII 53

HighPassBase

Iir::PoleFilter< HighPassBase, DirectFormII, 4 > 92

Iir::HighPassTransform 62

HighShelfBase

Iir::PoleFilter< HighShelfBase, DirectFormII, 4 > 92

Iir::Layout< MaxPoles > 73

Iir::LayoutBase 73

Iir::Butterworth::AnalogLowPass 14

Iir::Butterworth::AnalogLowShelf 15

Iir::ChebyshevI::AnalogLowPass 14

Iir::ChebyshevI::AnalogLowShelf 15

Iir::ChebyshevII::AnalogLowPass 14

Iir::ChebyshevII::AnalogLowShelf 15

LowPassBase

Iir::PoleFilter< LowPassBase, DirectFormII, 4 > 92

Iir::LowPassTransform 82

LowShelfBase

Iir::PoleFilter< LowShelfBase, DirectFormII, 4 > 92

Iir::PoleZeroPair 94

Iir::BiquadPoleState 49

Iir::Cascade::Storage 98

Iir::TransposedDirectFormII 98

4 Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Iir::RBJ::AllPass 12

Iir::Butterworth::AnalogLowPass 14

Iir::ChebyshevI::AnalogLowPass 14

Iir::ChebyshevII::AnalogLowPass 14

Iir::Butterworth::AnalogLowShelf 15

Iir::ChebyshevI::AnalogLowShelf 15

Iir::ChebyshevII::AnalogLowShelf 15

Iir::Butterworth::BandPass< FilterOrder, StateType > 16

Iir::ChebyshevI::BandPass< FilterOrder, StateType > 18

Iir::ChebyshevII::BandPass< FilterOrder, StateType > 20

Iir::RBJ::BandPass1 22

Iir::RBJ::BandPass2 23

Iir::Butterworth::BandPassBase 25

Iir::ChebyshevI::BandPassBase 25

Iir::ChebyshevII::BandPassBase 26

Iir::BandPassTransform 26

Iir::Butterworth::BandShelf< FilterOrder, StateType > 27

Iir::ChebyshevI::BandShelf< FilterOrder, StateType > 29

Iir::ChebyshevII::BandShelf< FilterOrder, StateType > 31

Iir::RBJ::BandShelf 33

Iir::Butterworth::BandShelfBase 35

Iir::ChebyshevI::BandShelfBase 36

Iir::ChebyshevII::BandShelfBase 36

Iir::Butterworth::BandStop< FilterOrder, StateType > 37

Iir::ChebyshevI::BandStop< FilterOrder, StateType > 38

Iir::ChebyshevII::BandStop< FilterOrder, StateType > 41

Iir::RBJ::BandStop 43

Iir::Butterworth::BandStopBase 44

Iir::ChebyshevI::BandStopBase 45

Iir::ChebyshevII::BandStopBase 45

Iir::BandStopTransform 46

Iir::Biquad 46

Iir::BiquadPoleState 49

Iir::Cascade 49

Iir::CascadeStages< MaxStages, StateType > 51

Iir::ComplexPair 52

Iir::DirectFormI 52

Iir::DirectFormII 53

Iir::Butterworth::HighPass< FilterOrder, StateType > 53

Iir::ChebyshevI::HighPass< FilterOrder, StateType > 55

Iir::ChebyshevII::HighPass< FilterOrder, StateType > 57

Iir::RBJ::HighPass 59

Iir::Butterworth::HighPassBase 60

Iir::ChebyshevI::HighPassBase 61

Iir::ChebyshevII::HighPassBase 61

Iir::HighPassTransform 62

Iir::Butterworth::HighShelf< FilterOrder, StateType > 62

Iir::ChebyshevI::HighShelf< FilterOrder, StateType > 64

iir::ChebyshevII::HighShelf< FilterOrder, StateType > 66

iir::RBJ::HighShelf 68

iir::Butterworth::HighShelfBase 70

iir::ChebyshevI::HighShelfBase 70

iir::ChebyshevII::HighShelfBase 71

iir::RBJ::IIRNotch 71

iir::Layout< MaxPoles > 73

iir::LayoutBase 73

iir::Butterworth::LowPass< FilterOrder, StateType > 73

iir::ChebyshevI::LowPass< FilterOrder, StateType > 75

iir::ChebyshevII::LowPass< FilterOrder, StateType > 77

iir::RBJ::LowPass 79

iir::Butterworth::LowPassBase 80

iir::ChebyshevI::LowPassBase 81

iir::ChebyshevII::LowPassBase 81

iir::LowPassTransform 82

iir::Butterworth::LowShelf< FilterOrder, StateType > 82

iir::ChebyshevI::LowShelf< FilterOrder, StateType > 84

iir::ChebyshevII::LowShelf< FilterOrder, StateType > 86

iir::RBJ::LowShelf 88

iir::Butterworth::LowShelfBase 90

iir::ChebyshevI::LowShelfBase 90

iir::ChebyshevII::LowShelfBase 91

iir::Custom::OnePole 91

iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles > 92

iir::PoleFilterBase< AnalogPrototype > 93

iir::PoleFilterBase2 93

iir::PoleZeroPair 94

iir::RBJ::RBJbase 95

iir::Custom::SOSCascade< NSOS, StateType > 96

iir::Cascade::Storage 98

iir::TransposedDirectFormII 98

Iir::Custom::TwoPole 98

5 File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

Iir.h 99

iir/Biquad.h 99

iir/Butterworth.h 100

iir/Cascade.h 105

iir/ChebyshevI.h 106

iir/ChebyshevII.h 112

iir/Common.h 117

iir/Custom.h 118

iir/Layout.h 118

iir/MathSupplement.h 120

iir/PoleFilter.h 121

iir/RBJ.h 123

iir/State.h 125

iir/Types.h 126

6 Namespace Documentation

6.1 Iir Namespace Reference

Namespaces

- namespace Butterworth
- namespace ChebyshevI
- namespace ChebyshevII
- namespace Custom

Classes

- class Biquad
- struct BiquadPoleState
- class Cascade
- class CascadeStages
- class LayoutBase
- class Layout
- class PoleFilterBase2
- class PoleFilterBase
- struct PoleFilter
- class LowPassTransform
- class HighPassTransform
- class BandPassTransform
- class BandStopTransform
- class DirectFormI
- class DirectFormII
- class TransposedDirectFormII
- struct ComplexPair
- struct PoleZeroPair

6.1.1 Detailed Description

"A Collection of Useful C++ Classes for Digital Signal Processing" By Vinnie Falco and Bernd Porr

Official project location: https://github.com/berndporr/iir1

See Documentation.cpp for contact information, notes, and bibliography.

License: MIT License (http://www.opensource.org/licenses/mit-license.php) Copyright (c) 2009 by Vinnie Falco Copyright (c) 2011 by Bernd Porr
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

"A Collection of Useful C++ Classes for Digital Signal Processing" By Vinnie Falco and Bernd Porr

Official project location: https://github.com/berndporr/iir1

See Documentation.txt for contact information, notes, and bibliography.

License: MIT License (http://www.opensource.org/licenses/mit-license.php) Copyright (c) 2009 by Vinnie Falco Copyright (c) 2011-2021 by Bernd Porr
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE

LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
"A Collection of Useful C++ Classes for Digital Signal Processing" By Vinnie Falco and Bernd Porr
Official project location: https://github.com/berndporr/iir1
See Documentation.cpp for contact information, notes, and bibliography.

License: MIT License (http://www.opensource.org/licenses/mit-license.php) Copyright (c) 2009 by Vinnie Falco Copyright (c) 2011-2021 by Bernd Porr
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
"A Collection of Useful C++ Classes for Digital Signal Processing" By Vinnie Falco and Bernd Porr
Official project location: https://github.com/berndporr/iir1
See Documentation.txt for contact information, notes, and bibliography.

License: MIT License (http://www.opensource.org/licenses/mit-license.php) Copyright (c) 2009 by Vinnie Falco Copyright (c) 2011 by Bernd Porr
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
"A Collection of Useful C++ Classes for Digital Signal Processing" By Vinnie Falco and Bernd Porr
Official project location: https://github.com/berndporr/iir1
See Documentation.cpp for contact information, notes, and bibliography.

License: MIT License (http://www.opensource.org/licenses/mit-license.php) Copyright (c) 2009 by Vinnie Falco Copyright (c) 2011 by Bernd Porr
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE. Describes a filter as a collection of poles and zeros along with normalization information to achieve a specified gain at a specified frequency. The poles and zeros may lie either in the s or the z plane.

6.2 Iir::Butterworth Namespace Reference

Classes

- class AnalogLowPass
- class AnalogLowShelf
- struct LowPassBase
- struct HighPassBase
- struct BandPassBase
- struct BandStopBase
- struct LowShelfBase
- struct HighShelfBase
- struct BandShelfBase
- struct LowPass
- struct HighPass
- struct BandPass
- struct BandStop
- struct LowShelf
- struct HighShelf
- struct BandShelf

6.2.1 Detailed Description

Filters with Butterworth response characteristics. The filter order is usually set via the template parameter which reserves the correct space and is then automatically passed to the setup function. Optionally one can also provde the filter order at setup time to force a lower order than the default one.

6.3 Iir::ChebyshevI Namespace Reference

Classes

- class AnalogLowPass
- class AnalogLowShelf
- struct LowPassBase
- struct HighPassBase
- struct BandPassBase
- struct BandStopBase
- struct LowShelfBase
- struct HighShelfBase
- struct BandShelfBase
- struct LowPass
- struct HighPass
- struct BandPass
- struct BandStop
- struct LowShelf
- struct HighShelf
- struct BandShelf

6.3.1 Detailed Description

Filters with Chebyshev response characteristics. The last parameter defines the passband ripple in decibel.

6.4 Iir::ChebyshevII Namespace Reference

Classes

- class AnalogLowPass
- class AnalogLowShelf
- struct LowPassBase
- struct HighPassBase
- struct BandPassBase
- struct BandStopBase
- struct LowShelfBase
- struct HighShelfBase
- struct BandShelfBase
- struct LowPass
- struct HighPass
- struct BandPass
- struct BandStop
- struct LowShelf
- struct HighShelf
- struct BandShelf

6.4.1 Detailed Description

Filters with ChebyshevII response characteristics. The last parameter defines the minimal stopband rejection requested. Generally there will be frequencies where the rejection is much better but this parameter guarantees that the rejection is at least as specified.

6.5 Iir::Custom Namespace Reference

Classes

- struct OnePole
- struct TwoPole
- struct SOSCascade

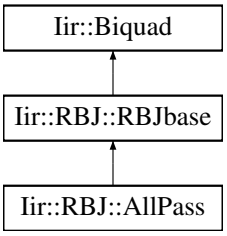
6.5.1 Detailed Description

Single pole, Biquad and cascade of Biquads with parameters allowing for directly setting the parameters.

7 Class Documentation

7.1 Iir::RBJ::AllPass Struct Reference

`#include <RBJ.h>`
Inheritance diagram for Iir::RBJ::AllPass:



Public Member Functions

- void setupN (double phaseFrequency, double q=(1/sqrt(2)))
- void setup (double sampleRate, double phaseFrequency, double q=(1/sqrt(2)))

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.1.1 Detailed Description

Allpass filter

7.1.2 Member Function Documentation

setup()

```
void Iir::RBJ::AllPass::setup (
             double sampleRate,
             double phaseFrequency,
             double q = (1/sqrt(2)))  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
phaseFrequency	Frequency where the phase flips
q	Q-factor

setupN()

```
void Iir::RBJ::AllPass::setupN (
             double phaseFrequency,
             double q = (1/sqrt(2)))
```

Calculates the coefficients

Parameters

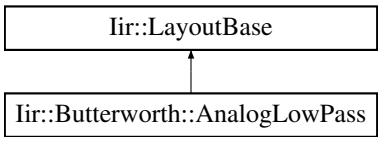
phaseFrequency	Normalised frequency where the phase flips
q	Q-factor

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.2 Iir::Butterworth::AnalogLowPass Class Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::AnalogLowPass:



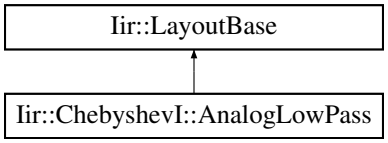
7.2.1 Detailed Description

Analogue lowpass prototypes (s-plane)
The documentation for this class was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.3 Iir::ChebyshevI::AnalogLowPass Class Reference

`#include <ChebyshevI.h>`
Inheritance diagram for Iir::ChebyshevI::AnalogLowPass:



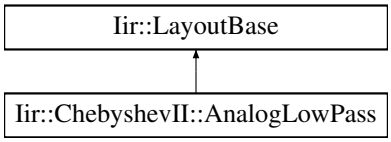
7.3.1 Detailed Description

Analog lowpass prototypes (s-plane)
The documentation for this class was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.4 Iir::ChebyshevII::AnalogLowPass Class Reference

`#include <ChebyshevII.h>`
Inheritance diagram for Iir::ChebyshevII::AnalogLowPass:



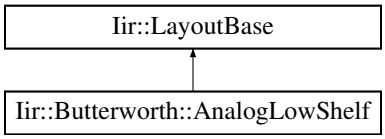
7.4.1 Detailed Description

Analogue lowpass prototype (s-plane)
The documentation for this class was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.5 Iir::Butterworth::AnalogLowShelf Class Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::AnalogLowShelf:



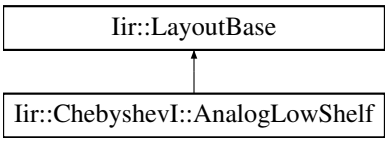
7.5.1 Detailed Description

Analogue low shelf prototypes (s-plane)
The documentation for this class was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.6 Iir::ChebyshevI::AnalogLowShelf Class Reference

`#include <ChebyshevI.h>`
Inheritance diagram for Iir::ChebyshevI::AnalogLowShelf:



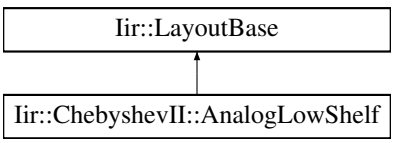
7.6.1 Detailed Description

Analog lowpass shelf prototype (s-plane)
The documentation for this class was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.7 Iir::ChebyshevII::AnalogLowShelf Class Reference

`#include <ChebyshevII.h>`
Inheritance diagram for Iir::ChebyshevII::AnalogLowShelf:



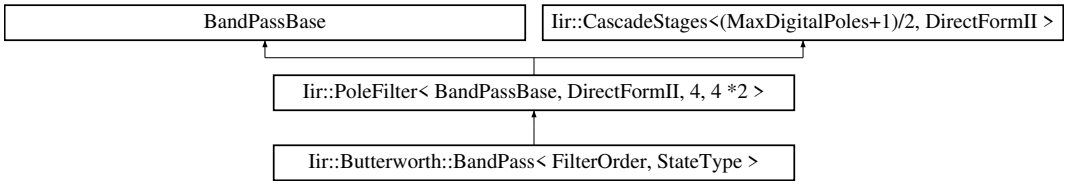
7.7.1 Detailed Description

Analogue shelf lowpass prototype (s-plane)
The documentation for this class was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.8 Iir::Butterworth::BandPass< FilterOrder, StateType > Struct Template Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::BandPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency)
- void setupN (double centerFrequency, double widthFrequency)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.8.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::Butterworth::BandPass< FilterOrder, StateType >

Butterworth Bandpass filter.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.8.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandPass< FilterOrder, StateType >::setup (
            double sampleRate,
            double centerFrequency,
            double widthFrequency)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

sampleRate	Sampling rate
centerFrequency	Centre frequency of the bandpass
widthFrequency	Width of the bandpass

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandPass< FilterOrder, StateType >::setup (
            int reqOrder,
            double sampleRate,
            double centerFrequency,
            double widthFrequency)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
sampleRate	Sampling rate
centerFrequency	Centre frequency of the bandpass
widthFrequency	Width of the bandpass

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandPass< FilterOrder, StateType >::setupN (
            double centerFrequency,
            double widthFrequency)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the bandpass
widthFrequency	Width of the bandpass in normalised freq

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandPass< FilterOrder, StateType >::setupN (
            int reqOrder,
            double centerFrequency,
            double widthFrequency)  [inline]
```

Calculates the coefficients

Parameters

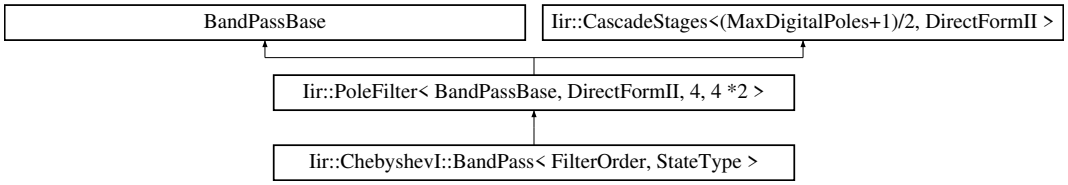
reqOrder	The actual order which can be less than the instantiated one
centerFrequency	Normalised centre frequency (0..1/2) of the bandpass
widthFrequency	Width of the bandpass in normalised freq

The documentation for this struct was generated from the following file:

- iir/Butterworth.h

7.9 Iir::ChebyshevI::BandPass< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevI.h>`
Inheritance diagram for Iir::ChebyshevI::BandPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency, double rippleDb)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency, double rippleDb)
- void setupN (double centerFrequency, double widthFrequency, double rippleDb)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency, double rippleDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.9.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevI::BandPass< FilterOrder, StateType >

ChebyshevI bandpass filter

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.9.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandPass< FilterOrder, StateType >::setup (
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
widthFrequency	Frequency with of the passband
rippleDb	Permitted ripples in dB in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandPass< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double centerFrequency,
             double widthFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
widthFrequency	Frequency with of the passband
rippleDb	Permitted ripples in dB in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandPass< FilterOrder, StateType >::setupN (
             double centerFrequency,
             double widthFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

centerFrequency	Normalised center frequency (0..1/2) of the bandpass
widthFrequency	Frequency with of the passband
rippleDb	Permitted ripples in dB in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandPass< FilterOrder, StateType >::setupN (
             int reqOrder,
             double centerFrequency,
             double widthFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

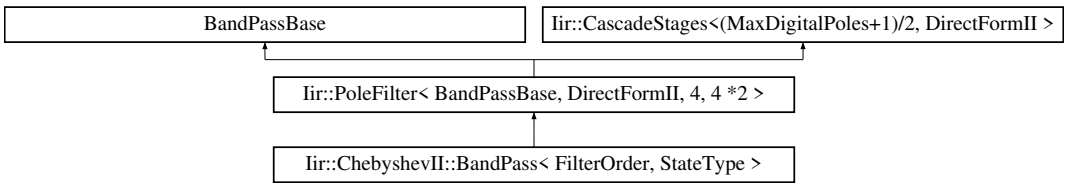
reqOrder	Actual order for the filter calculations
centerFrequency	Normalised center frequency (0..1/2) of the bandpass
widthFrequency	Frequency with of the passband
rippleDb	Permitted ripples in dB in the passband

The documentation for this struct was generated from the following file:

- iir/ChebyshevI.h

7.10 Iir::ChebyshevII::BandPass< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevII.h>`
Inheritance diagram for Iir::ChebyshevII::BandPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency, double stopBandDb)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency, double stop↩ BandDb)
- void setupN (double centerFrequency, double widthFrequency, double stopBandDb)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency, double stopBandDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.10.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevII::BandPass< FilterOrder, StateType >

ChebyshevII bandpass filter

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.10.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandPass< FilterOrder, StateType >::setup (
```

```
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
widthFrequency	Width of the bandpass
stopBandDb	Permitted ripples in dB in the stopband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandPass< FilterOrder, StateType >::setup (
            int reqOrder,
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
widthFrequency	Width of the bandpass
stopBandDb	Permitted ripples in dB in the stopband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandPass< FilterOrder, StateType >::setupN (
            double centerFrequency,
            double widthFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the bandpass
widthFrequency	Width of the bandpass
stopBandDb	Permitted ripples in dB in the stopband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandPass< FilterOrder, StateType >::setupN (
```

```
        int reqOrder,
        double centerFrequency,
        double widthFrequency,
        double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
centerFrequency	Normalised centre frequency (0..1/2) of the bandpass
widthFrequency	Width of the bandpass
stopBandDb	Permitted ripples in dB in the stopband

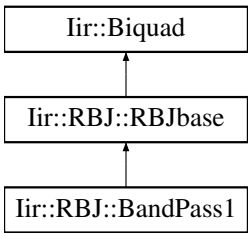
The documentation for this struct was generated from the following file:

- iir/ChebyshevII.h

7.11 Iir::RBJ::BandPass1 Struct Reference

`#include <RBJ.h>`

Inheritance diagram for Iir::RBJ::BandPass1:



Public Member Functions

- void setupN (double centerFrequency, double bandWidth)
- void setup (double sampleRate, double centerFrequency, double bandWidth)

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const

- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.11.1 Detailed Description

Bandpass with constant skirt gain

7.11.2 Member Function Documentation

setup()

```
void Iir::RBJ::BandPass1::setup (
             double sampleRate,
             double centerFrequency,
             double bandWidth)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
bandWidth	Bandwidth in octaves

setupN()

```
void Iir::RBJ::BandPass1::setupN (
             double centerFrequency,
             double bandWidth)
```

Calculates the coefficients

Parameters

centerFrequency	Center frequency of the bandpass
bandWidth	Bandwidth in octaves

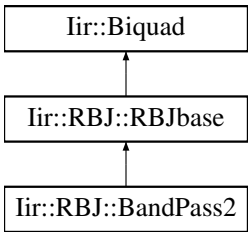
The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.12 Iir::RBJ::BandPass2 Struct Reference

```
#include <RBJ.h>
```

Inheritance diagram for Iir::RBJ::BandPass2:



Public Member Functions

- void setupN (double centerFrequency, double bandWidth)
- void setup (double sampleRate, double centerFrequency, double bandWidth)

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.12.1 Detailed Description

Bandpass with constant 0 dB peak gain

7.12.2 Member Function Documentation

setup()

```
void Iir::RBJ::BandPass2::setup (
            double sampleRate,
            double centerFrequency,
            double bandWidth)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
bandWidth	Bandwidth in octaves

setupN()

```
void Iir::RBJ::BandPass2::setupN (
            double centerFrequency,
            double bandWidth)
```

Calculates the coefficients

Parameters

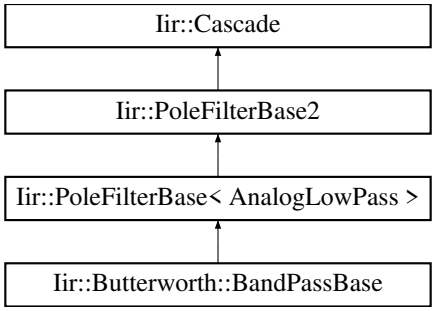
centerFrequency	Normalised centre frequency of the bandpass
bandWidth	Bandwidth in octaves

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.13 Iir::Butterworth::BandPassBase Struct Reference

Inheritance diagram for Iir::Butterworth::BandPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

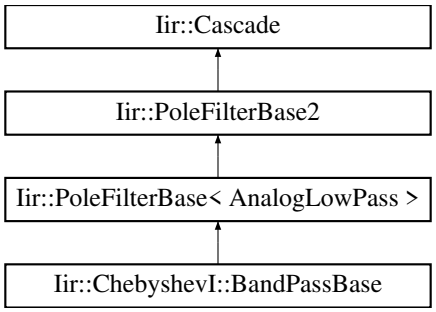
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.14 Iir::ChebyshevI::BandPassBase Struct Reference

Inheritance diagram for Iir::ChebyshevI::BandPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

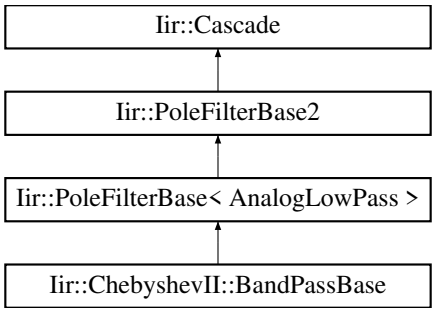
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.15 Iir::ChebyshevII::BandPassBase Struct Reference

Inheritance diagram for Iir::ChebyshevII::BandPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.16 Iir::BandPassTransform Class Reference

`#include <PoleFilter.h>`

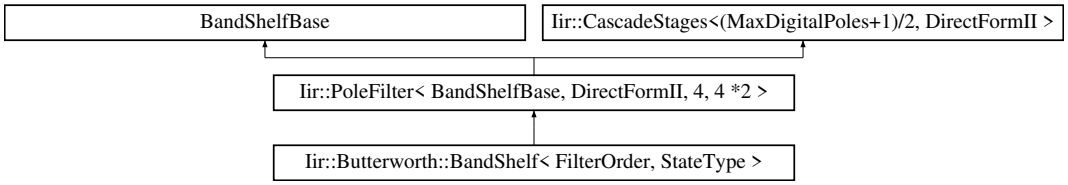
7.16.1 Detailed Description

low pass to band pass transform
The documentation for this class was generated from the following files:

- iir/PoleFilter.h
- iir/PoleFilter.cpp

7.17 Iir::Butterworth::BandShelf< FilterOrder, StateType > Struct Template Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::BandShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency, double gainDb)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency, double gainDb)
- void setupN (double centerFrequency, double widthFrequency, double gainDb)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency, double gainDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.17.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::Butterworth::BandShelf< FilterOrder, StateType >

Butterworth Bandshelf filter: it is a bandpass filter which amplifies at a specified gain in dB the frequencies in the passband.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.17.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandShelf< FilterOrder, StateType >::setup (
            double sampleRate,
            double centerFrequency,
```

```
            double widthFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

sampleRate	Sampling rate
centerFrequency	Centre frequency of the passband
widthFrequency	Width of the passband
gainDb	The gain in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandShelf< FilterOrder, StateType >::setup (
            int reqOrder,
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
sampleRate	Sampling rate
centerFrequency	Centre frequency of the passband
widthFrequency	Width of the passband
gainDb	The gain in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandShelf< FilterOrder, StateType >::setupN (
            double centerFrequency,
            double widthFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the passband
widthFrequency	Width of the passband
gainDb	The gain in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandShelf< FilterOrder, StateType >::setupN (
            int reqOrder,
```

```
            double centerFrequency,
            double widthFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
centerFrequency	Normalised centre frequency (0..1/2) of the passband
widthFrequency	Width of the passband
gainDb	The gain in the passband

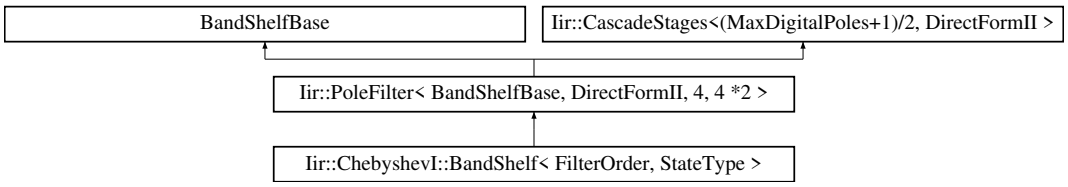
The documentation for this struct was generated from the following file:

- iir/Butterworth.h

7.18 Iir::ChebyshevI::BandShelf< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevI.h>`

Inheritance diagram for Iir::ChebyshevI::BandShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency, double gainDb, double rippleDb)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency, double gainDb, double rippleDb)
- void setupN (double centerFrequency, double widthFrequency, double gainDb, double rippleDb)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency, double gainDb, double rippleDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.18.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevI::BandShelf< FilterOrder, StateType >

ChebyshevI bandshelf filter. Specified gain in the passband. Otherwise 0 dB.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.18.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandShelf< FilterOrder, StateType >::setup (
             double sampleRate,
             double centerFrequency,
             double widthFrequency,
             double gainDb,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the passband
widthFrequency	Width of the passband.
gainDb	Gain in the passband. The stopband has 0 dB.
rippleDb	Permitted ripples in dB in the passband.

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandShelf< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double centerFrequency,
             double widthFrequency,
             double gainDb,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
sampleRate	Sampling rate
centerFrequency	Center frequency of the passband
widthFrequency	Width of the passband.
gainDb	Gain in the passband. The stopband has 0 dB.
rippleDb	Permitted ripples in dB in the passband.

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandShelf< FilterOrder, StateType >::setupN (
             double centerFrequency,
             double widthFrequency,
             double gainDb,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the passband
widthFrequency	Width of the passband.
gainDb	Gain in the passband. The stopband has 0 dB.
rippleDb	Permitted ripples in dB in the passband.

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandShelf< FilterOrder, StateType >::setupN (
            int reqOrder,
            double centerFrequency,
            double widthFrequency,
            double gainDb,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
centerFrequency	Normalised centre frequency (0..1/2) of the passband
widthFrequency	Width of the passband.
gainDb	Gain in the passband. The stopband has 0 dB.
rippleDb	Permitted ripples in dB in the passband.

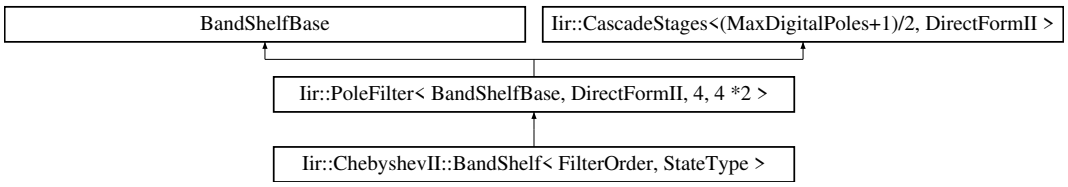
The documentation for this struct was generated from the following file:

- iir/ChebyshevI.h

7.19 Iir::ChebyshevII::BandShelf< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevII.h>`

Inheritance diagram for Iir::ChebyshevII::BandShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency, double gainDb, double stopBandDb)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency, double gain↩ Db, double stopBandDb)
- void setupN (double centerFrequency, double widthFrequency, double gainDb, double stopBandDb)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency, double gainDb, double stop↩ BandDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.19.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevII::BandShelf< FilterOrder, StateType >

ChebyshevII bandshelf filter. Bandpass with specified gain and 0 dB gain in the stopband.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.19.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandShelf< FilterOrder, StateType >::setup (
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double gainDb,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
widthFrequency	Width of the bandpass
gainDb	Gain in the passband. The stopband has always 0dB.
stopBandDb	Permitted ripples in dB in the stopband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandShelf< FilterOrder, StateType >::setup (
            int reqOrder,
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double gainDb,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
sampleRate	Sampling rate
centerFrequency	Center frequency of the bandpass
widthFrequency	Width of the bandpass
gainDb	Gain in the passband. The stopband has always 0dB.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandShelf< FilterOrder, StateType >::setupN (
            double centerFrequency,
            double widthFrequency,
            double gainDb,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the bandpass
widthFrequency	Width of the bandpass
gainDb	Gain in the passband. The stopband has always 0dB.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandShelf< FilterOrder, StateType >::setupN (
            int reqOrder,
            double centerFrequency,
            double widthFrequency,
            double gainDb,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
centerFrequency	Normalised centre frequency (0..1/2) of the bandpass
widthFrequency	Width of the bandpass
gainDb	Gain in the passband. The stopband has always 0dB.
stopBandDb	Permitted ripples in dB in the stopband

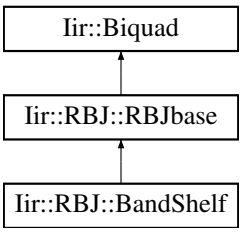
The documentation for this struct was generated from the following file:

- iir/ChebyshevII.h

7.20 Iir::RBJ::BandShelf Struct Reference

`#include <RBJ.h>`

Inheritance diagram for Iir::RBJ::BandShelf:



Public Member Functions

- void setupN (double centerFrequency, double gainDb, double bandWidth)
- void setup (double sampleRate, double centerFrequency, double gainDb, double bandWidth)

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.20.1 Detailed Description

Band shelf: 0db in the stopband and gainDb in the passband.

7.20.2 Member Function Documentation

setup()

```
void Iir::RBJ::BandShelf::setup (
            double sampleRate,
            double centerFrequency,
            double gainDb,
            double bandWidth)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
centerFrequency	frequency
gainDb	Gain in the passband
bandWidth	Bandwidth in octaves

setupN()

```
void Iir::RBJ::BandShelf::setupN (
            double centerFrequency,
            double gainDb,
            double bandWidth)
```

Calculates the coefficients

Parameters

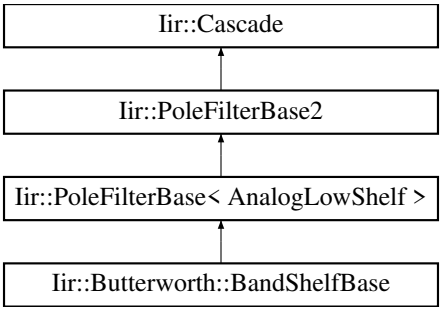
centerFrequency	Normalised centre frequency
gainDb	Gain in the passband
bandWidth	Bandwidth in octaves

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.21 Iir::Butterworth::BandShelfBase Struct Reference

Inheritance diagram for Iir::Butterworth::BandShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

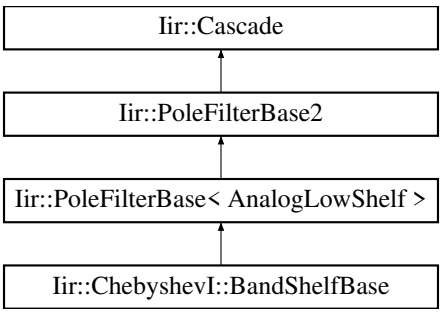
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.22 Iir::ChebyshevI::BandShelfBase Struct Reference

Inheritance diagram for Iir::ChebyshevI::BandShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

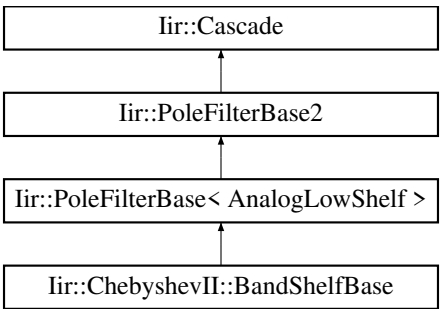
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.23 Iir::ChebyshevII::BandShelfBase Struct Reference

Inheritance diagram for Iir::ChebyshevII::BandShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

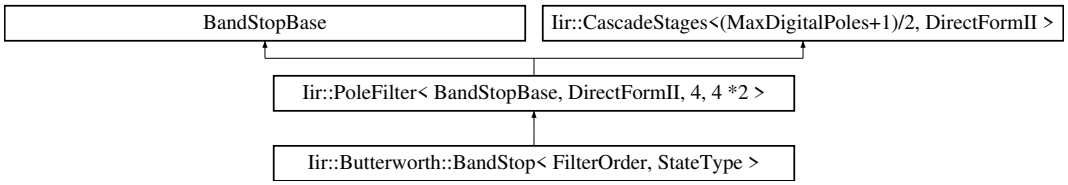
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.24 Iir::Butterworth::BandStop< FilterOrder, StateType > Struct Template Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::BandStop< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency)
- void setupN (double centerFrequency, double widthFrequency)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.24.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::Butterworth::BandStop< FilterOrder, StateType >

Butterworth Bandstop filter.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.24.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandStop< FilterOrder, StateType >::setup (
            double sampleRate,
            double centerFrequency,
            double widthFrequency)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

sampleRate	Sampling rate
centerFrequency	Centre frequency of the bandstop
widthFrequency	Width of the bandstop

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandStop< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double centerFrequency,
             double widthFrequency)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
sampleRate	Sampling rate
centerFrequency	Centre frequency of the bandstop
widthFrequency	Width of the bandstop

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandStop< FilterOrder, StateType >::setupN (
             double centerFrequency,
             double widthFrequency)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the bandstop
widthFrequency	Normalised width of the bandstop

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::BandStop< FilterOrder, StateType >::setupN (
             int reqOrder,
             double centerFrequency,
             double widthFrequency)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
centerFrequency	Normalised centre frequency (0..1/2) of the bandstop
widthFrequency	Normalised width of the bandstop

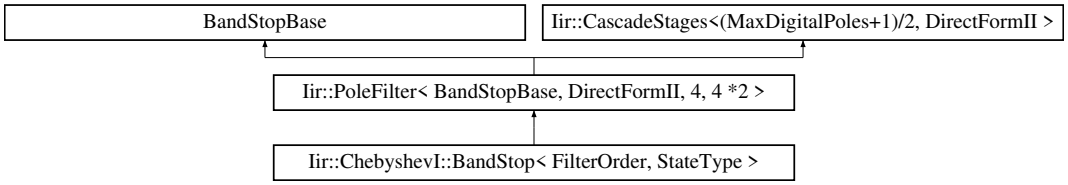
The documentation for this struct was generated from the following file:

- iir/Butterworth.h

7.25 Iir::ChebyshevI::BandStop< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevI.h>`

Inheritance diagram for Iir::ChebyshevI::BandStop< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency, double rippleDb)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency, double rippleDb)
- void setupN (double centerFrequency, double widthFrequency, double rippleDb)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency, double rippleDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.25.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevI::BandStop< FilterOrder, StateType >

ChebyshevI bandstop filter

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.25.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandStop< FilterOrder, StateType >::setup (
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the notch
widthFrequency	Frequency with of the notch
rippleDb	Permitted ripples in dB in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandStop< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double centerFrequency,
             double widthFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
sampleRate	Sampling rate
centerFrequency	Center frequency of the notch
widthFrequency	Frequency with of the notch
rippleDb	Permitted ripples in dB in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandStop< FilterOrder, StateType >::setupN (
             double centerFrequency,
             double widthFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the notch
widthFrequency	Frequency width of the notch
rippleDb	Permitted ripples in dB in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::BandStop< FilterOrder, StateType >::setupN (
             int reqOrder,
             double centerFrequency,
             double widthFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

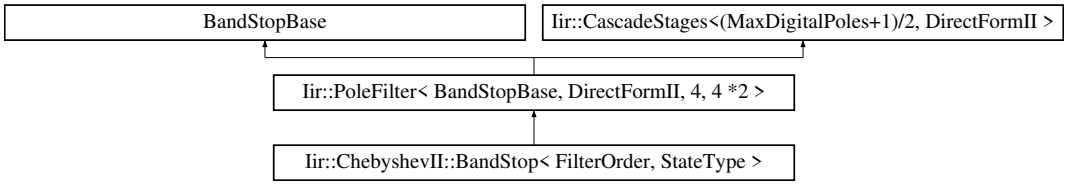
reqOrder	Actual order for the filter calculations
centerFrequency	Normalised centre frequency (0..1/2) of the notch
widthFrequency	Frequency width of the notch
rippleDb	Permitted ripples in dB in the passband

The documentation for this struct was generated from the following file:

- iir/ChebyshevI.h

7.26 Iir::ChebyshevII::BandStop< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevII.h>`
Inheritance diagram for Iir::ChebyshevII::BandStop< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double centerFrequency, double widthFrequency, double stopBandDb)
- void setup (int reqOrder, double sampleRate, double centerFrequency, double widthFrequency, double stopBandDb)
- void setupN (double centerFrequency, double widthFrequency, double stopBandDb)
- void setupN (int reqOrder, double centerFrequency, double widthFrequency, double stopBandDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxDigitalPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.26.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevII::BandStop< FilterOrder, StateType >

ChebyshevII bandstop filter.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.26.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandStop< FilterOrder, StateType >::setup (
            double sampleRate,
            double centerFrequency,
            double widthFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the bandstop
widthFrequency	Width of the bandstop
stopBandDb	Permitted ripples in dB in the stopband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandStop< FilterOrder, StateType >::setup (
              int reqOrder,
              double sampleRate,
              double centerFrequency,
              double widthFrequency,
              double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
sampleRate	Sampling rate
centerFrequency	Center frequency of the bandstop
widthFrequency	Width of the bandstop
stopBandDb	Permitted ripples in dB in the stopband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandStop< FilterOrder, StateType >::setupN (
              double centerFrequency,
              double widthFrequency,
              double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

centerFrequency	Normalised centre frequency (0..1/2) of the bandstop
widthFrequency	Width of the bandstop
stopBandDb	Permitted ripples in dB in the stopband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::BandStop< FilterOrder, StateType >::setupN (
              int reqOrder,
              double centerFrequency,
              double widthFrequency,
              double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

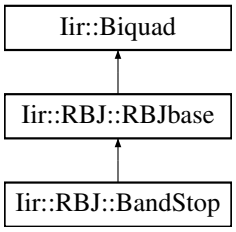
reqOrder	Requested order which can be less than the instantiated one
centerFrequency	Normalised centre frequency (0..1/2) of the bandstop
widthFrequency	Width of the bandstop
stopBandDb	Permitted ripples in dB in the stopband

The documentation for this struct was generated from the following file:

- iir/ChebyshevII.h

7.27 Iir::RBJ::BandStop Struct Reference

`#include <RBJ.h>`
Inheritance diagram for Iir::RBJ::BandStop:



Public Member Functions

- void setupN (double centerFrequency, double bandWidth)
- void setup (double sampleRate, double centerFrequency, double bandWidth)

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)
  filter operation
- void **reset** ()
  resets the delay lines to zero
- const DirectFormI & **getState** ()
  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.27.1 Detailed Description

Bandstop filter. Warning: the bandwidth might not be accurate for narrow notches.

7.27.2 Member Function Documentation

setup()

```
void Iir::RBJ::BandStop::setup (
            double sampleRate,
            double centerFrequency,
            double bandWidth)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the bandstop
bandWidth	Bandwidth in octaves

setupN()

```
void Iir::RBJ::BandStop::setupN (
            double centerFrequency,
            double bandWidth)
```

Calculates the coefficients

Parameters

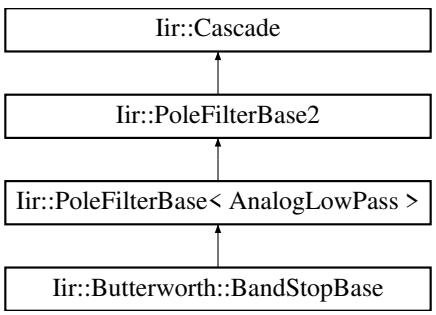
centerFrequency	Normalised Centre frequency of the bandstop
bandWidth	Bandwidth in octaves

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.28 Iir::Butterworth::BandStopBase Struct Reference

Inheritance diagram for Iir::Butterworth::BandStopBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

- int getNumStages () const

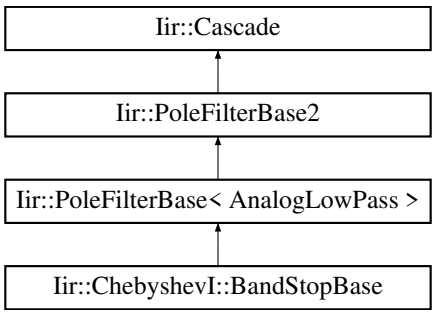
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.29 Iir::ChebyshevI::BandStopBase Struct Reference

Inheritance diagram for Iir::ChebyshevI::BandStopBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

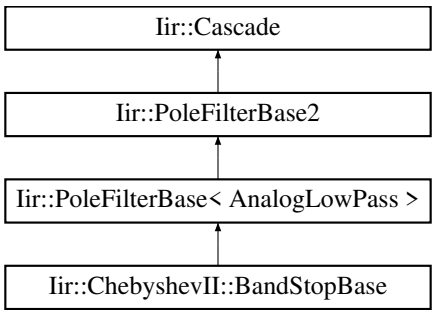
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.30 Iir::ChebyshevII::BandStopBase Struct Reference

Inheritance diagram for Iir::ChebyshevII::BandStopBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

- int getNumStages () const
- const Biquad & operator[ ] (int index)

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.31 Iir::BandStopTransform Class Reference

`#include <PoleFilter.h>`

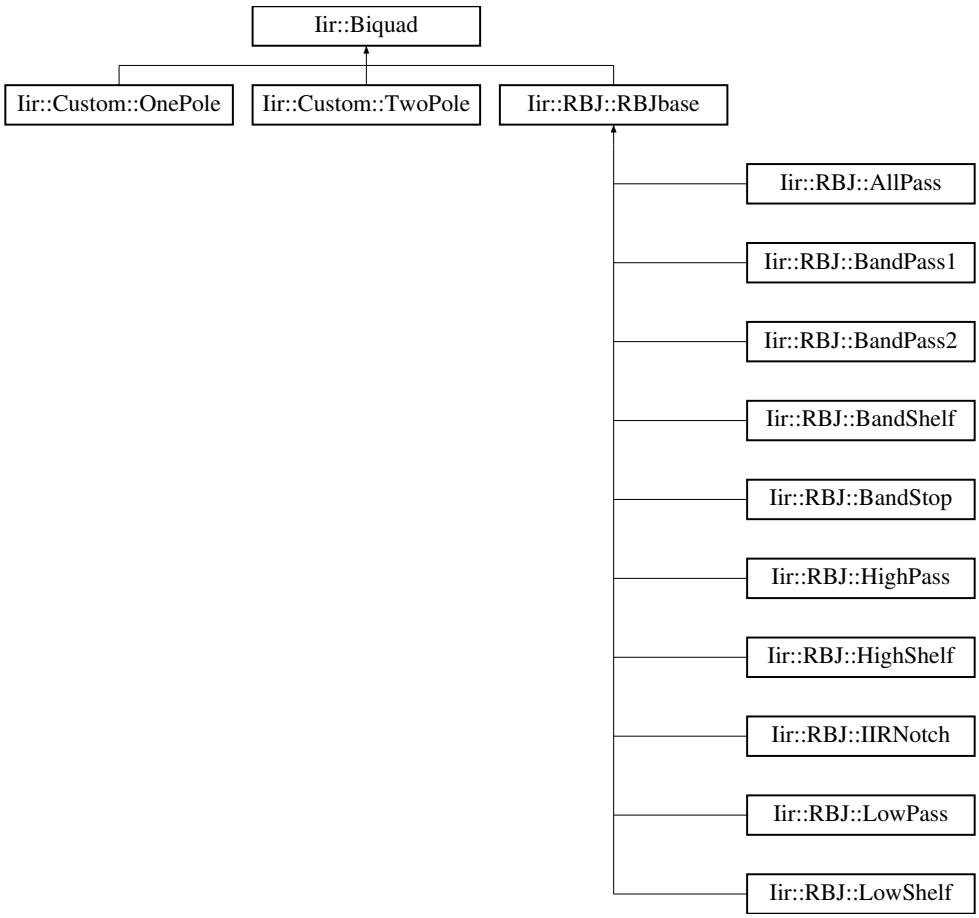
7.31.1 Detailed Description

low pass to band stop transform
The documentation for this class was generated from the following files:

- iir/PoleFilter.h
- iir/PoleFilter.cpp

7.32 Iir::Biquad Class Reference

Inheritance diagram for Iir::Biquad:



Public Member Functions

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const

- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.32.1 Member Function Documentation

applyScale()

```
void Iir::Biquad::applyScale (
            double scale)
```

Performs scaling operation on the FIR coefficients

Parameters

scale	Mulitplies the coefficients b0,b1,b2 with the scaling factor scale.

filter()

```
template<class StateType>
double Iir::Biquad::filter (
            double s,
            StateType & state) const  [inline]
```

Filter a sample with the coefficients provided here and the State provided as an argument.

Parameters

s	The sample to be filtered.
state	The Delay lines (instance of a state from State.h)

Returns

The filtered sample.

getA0()

```
double Iir::Biquad::getA0 () const  [inline]
```

Returns 1st IIR coefficient (usually one)

getA1()

```
double Iir::Biquad::getA1 () const  [inline]
```

Returns 2nd IIR coefficient

getA2()

```
double Iir::Biquad::getA2 () const  [inline]
```
Returns 3rd IIR coefficient

getB0()

```
double Iir::Biquad::getB0 () const  [inline]
```
Returns 1st FIR coefficient

getB1()

```
double Iir::Biquad::getB1 () const  [inline]
```
Returns 2nd FIR coefficient

getB2()

```
double Iir::Biquad::getB2 () const  [inline]
```
Returns 3rd FIR coefficient

getPoleZeros()

```
std::vector< PoleZeroPair > Iir::Biquad::getPoleZeros () const
```
Returns the pole / zero Pairs as a vector.

response()

```
complex_t Iir::Biquad::response (
            double normalizedFrequency) const
```
Calculate filter response at the given normalized frequency and return the complex response.
Gets the frequency response of the Biquad

Parameters

normalizedFrequency	Normalised frequency (0 to 0.5)

setCoefficients()

```
void Iir::Biquad::setCoefficients (
            double a0,
            double a1,
            double a2,
            double b0,
            double b1,
            double b2)
```
Sets all coefficients

Parameters

a0	1st IIR coefficient
a1	2nd IIR coefficient
a2	3rd IIR coefficient
b0	1st FIR coefficient
b1	2nd FIR coefficient
b2	3rd FIR coefficient

setIdentity()

```
void Iir::Biquad::setIdentity ()
```
Sets the coefficiens as pass through. (b0=1,a0=1, rest zero)

setOnePole()

```
void Iir::Biquad::setOnePole (
            complex_t pole,
            complex_t zero)
```
Sets one (real) pole and zero. Throws exception if imaginary components.

setPoleZeroPair()

```
void Iir::Biquad::setPoleZeroPair (
            const PoleZeroPair & pair)  [inline]
```
Sets a complex conjugate pair

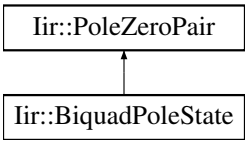
setTwoPole()

```
void Iir::Biquad::setTwoPole (
            complex_t pole1,
            complex_t zero1,
            complex_t pole2,
            complex_t zero2)
```
Sets two poles/zoes as a pair. Needs to be complex conjugate.
The documentation for this class was generated from the following files:

- iir/Biquad.h
- iir/Biquad.cpp

7.33 Iir::BiquadPoleState Struct Reference

`#include <Biquad.h>`
Inheritance diagram for Iir::BiquadPoleState:



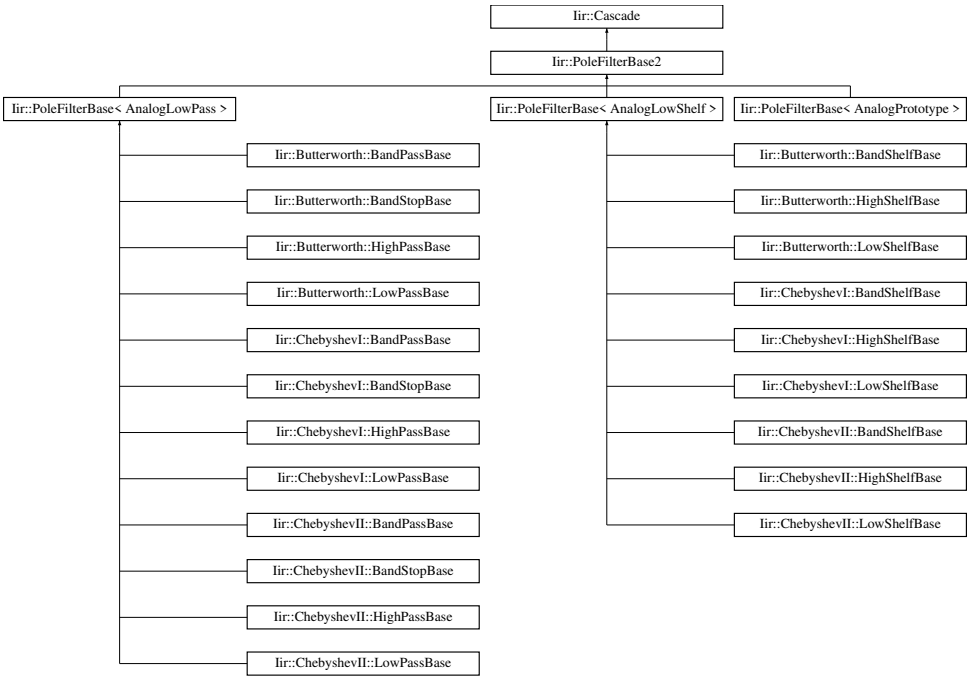
7.33.1 Detailed Description

Expresses a biquad as a pair of pole/zeros, with gain values so that the coefficients can be reconstructed precisely.
The documentation for this struct was generated from the following files:

- iir/Biquad.h
- iir/Biquad.cpp

7.34 Iir::Cascade Class Reference

`#include <Cascade.h>`
Inheritance diagram for Iir::Cascade:



Classes

- struct Storage

Public Member Functions

- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

7.34.1 Detailed Description

Holds coefficients for a cascade of second order sections.

7.34.2 Member Function Documentation

getNumStages()

```
int Iir::Cascade::getNumStages () const  [inline]
```

Returns the number of Biquads kept here

getPoleZeros()

```
std::vector< PoleZeroPair > Iir::Cascade::getPoleZeros () const
```

Returns a vector with all pole/zero pairs of the whole Biqad cascade

operator[]()

```
const Biquad & Iir::Cascade::operator[] (
            int index)  [inline]
```

Returns a reference to a biquad

response()

```
complex_t Iir::Cascade::response (
            double normalizedFrequency) const
```

Calculate filter response at the given normalized frequency

Parameters

normalizedFrequency	Frequency from 0 to 0.5 (Nyquist)

The documentation for this class was generated from the following files:

- iir/Cascade.h
- iir/Cascade.cpp

7.35 Iir::CascadeStages< MaxStages, StateType > Class Template Reference

```
#include <Cascade.h>
```

Public Member Functions

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- template<typename Sample>
  Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.35.1 Detailed Description

template<int MaxStages, class StateType>
class Iir::CascadeStages< MaxStages, StateType >

Storage for Cascade: This holds a chain of 2nd order filters with its coefficients.

7.35.2 Member Function Documentation

filter()

```
template<int MaxStages, class StateType>
template<typename Sample>
Sample Iir::CascadeStages< MaxStages, StateType >::filter (
            const Sample in)  [inline]
```

Filters one sample through the whole chain of biquads and return the result

Parameters

in	Sample to be filtered

Returns

filtered sample

getCascadeStorage()

```
template<int MaxStages, class StateType>
const Cascade::Storage Iir::CascadeStages< MaxStages, StateType >::getCascadeStorage ()  [inline]
```

Returns the coefficients of the entire Biquad chain

reset()

```
template<int MaxStages, class StateType>
void Iir::CascadeStages< MaxStages, StateType >::reset ()  [inline]
```

Resets all biquads (i.e. the delay lines but not the coefficients)

setup()

```
template<int MaxStages, class StateType>
void Iir::CascadeStages< MaxStages, StateType >::setup (
            const double(&) sosCoefficients[MaxStages][6])  [inline]
```

Sets the coefficients of the whole chain of biquads.

Parameters

sosCoefficients	2D array in Python style sos ordering: 0-2: FIR, 3-5: IIR coeff.

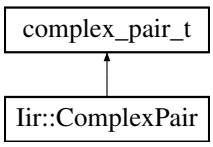
The documentation for this class was generated from the following file:

- iir/Cascade.h

7.36 Iir::ComplexPair Struct Reference

```
#include <Types.h>
```

Inheritance diagram for Iir::ComplexPair:



Public Member Functions

- bool isMatchedPair () const

7.36.1 Detailed Description

A conjugate or real pair

7.36.2 Member Function Documentation

isMatchedPair()

```
bool Iir::ComplexPair::isMatchedPair () const  [inline]
```

Returns true if this is either a conjugate pair, or a pair of reals where neither is zero.
The documentation for this struct was generated from the following file:

- iir/Types.h

7.37 Iir::DirectFormI Class Reference

```
#include <State.h>
```

7.37.1 Detailed Description

State for applying a second order section to a sample using Direct Form I
Difference equation:
y[n] = (b0/a0)∗x[n] + (b1/a0)∗x[n-1] + (b2/a0)∗x[n-2]

- (a1/a0)∗y[n-1] - (a2/a0)∗y[n-2]

The documentation for this class was generated from the following file:

- iir/State.h

7.38 Iir::DirectFormII Class Reference

`#include <State.h>`

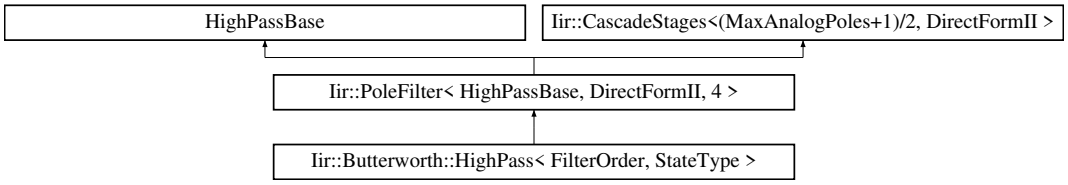
7.38.1 Detailed Description

State for applying a second order section to a sample using Direct Form II
Difference equation:
v[n] = x[n] - (a1/a0)∗v[n-1] - (a2/a0)∗v[n-2] y(n) = (b0/a0)∗v[n] + (b1/a0)∗v[n-1] + (b2/a0)∗v[n-2]
The documentation for this class was generated from the following file:

- iir/State.h

7.39 Iir::Butterworth::HighPass< FilterOrder, StateType > Struct Template Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::HighPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency)
- void setupN (double cutoffFrequency)
- void setupN (int reqOrder, double cutoffFrequency)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.39.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::Butterworth::HighPass< FilterOrder, StateType >

Butterworth Highpass filter.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.39.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighPass< FilterOrder, StateType >::setup (
            double sampleRate,
            double cutoffFrequency)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighPass< FilterOrder, StateType >::setup (
            int reqOrder,
            double sampleRate,
            double cutoffFrequency)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighPass< FilterOrder, StateType >::setupN (
            double cutoffFrequency)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighPass< FilterOrder, StateType >::setupN (
            int reqOrder,
            double cutoffFrequency)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one

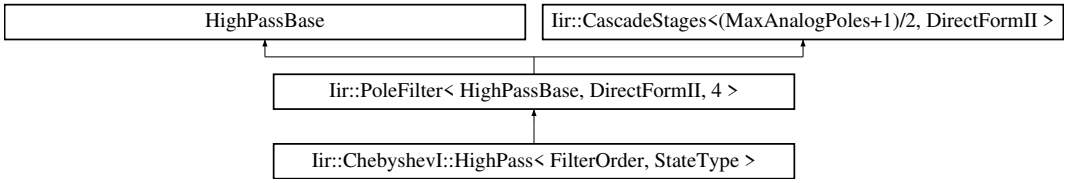
cutoffFrequency	Normalised cutoff frequency (0..1/2)

The documentation for this struct was generated from the following file:

- iir/Butterworth.h

7.40 Iir::ChebyshevI::HighPass< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevI.h>`
Inheritance diagram for Iir::ChebyshevI::HighPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double rippleDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double rippleDb)
- void setupN (double cutoffFrequency, double rippleDb)
- void setupN (int reqOrder, double cutoffFrequency, double rippleDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.40.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevI::HighPass< FilterOrder, StateType >

ChebyshevI highpass filter

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.40.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighPass< FilterOrder, StateType >::setup (
            double sampleRate,
            double cutoffFrequency,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
rippleDb	Permitted ripples in dB in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighPass< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
rippleDb	Permitted ripples in dB in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighPass< FilterOrder, StateType >::setupN (
             double cutoffFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
rippleDb	Permitted ripples in dB in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighPass< FilterOrder, StateType >::setupN (
             int reqOrder,
             double cutoffFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

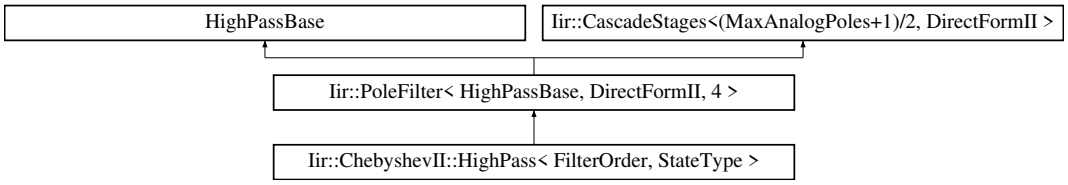
reqOrder	Actual order for the filter calculations
cutoffFrequency	Normalised cutoff frequency (0..1/2)
rippleDb	Permitted ripples in dB in the passband

The documentation for this struct was generated from the following file:

- iir/ChebyshevI.h

7.41 Iir::ChebyshevII::HighPass< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevII.h>`
Inheritance diagram for Iir::ChebyshevII::HighPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double stopBandDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double stopBandDb)
- void setupN (double cutoffFrequency, double stopBandDb)
- void setupN (int reqOrder, double cutoffFrequency, double stopBandDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.41.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevII::HighPass< FilterOrder, StateType >

ChebyshevII highpass filter

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.41.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighPass< FilterOrder, StateType >::setup (
            double sampleRate,
            double cutoffFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

sampleRate	Sampling rate

cutoffFrequency	Cutoff frequency.
stopBandDb	Permitted ripples in dB in the stopband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighPass< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighPass< FilterOrder, StateType >::setupN (
             double cutoffFrequency,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
stopBandDb	Permitted ripples in dB in the stopband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighPass< FilterOrder, StateType >::setupN (
             int reqOrder,
             double cutoffFrequency,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

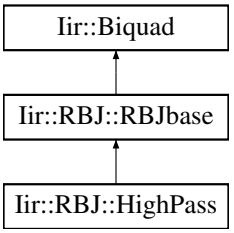
reqOrder	Requested order which can be less than the instantiated one
cutoffFrequency	Normalised cutoff frequency (0..1/2)
stopBandDb	Permitted ripples in dB in the stopband

The documentation for this struct was generated from the following file:

- iir/ChebyshevII.h

7.42 Iir::RBJ::HighPass Struct Reference

`#include <RBJ.h>`
Inheritance diagram for Iir::RBJ::HighPass:



Public Member Functions

- void setupN (double cutoffFrequency, double q=(1/sqrt(2)))
- void setup (double sampleRate, double cutoffFrequency, double q=(1/sqrt(2)))

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.42.1 Detailed Description

Highpass.

7.42.2 Member Function Documentation

setup()

```
void Iir::RBJ::HighPass::setup (
            double sampleRate,
            double cutoffFrequency,
            double q = (1/sqrt(2)))  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency
q	Q factor determines the resonance peak at the cutoff.

setupN()

```
void Iir::RBJ::HighPass::setupN (
            double cutoffFrequency,
            double q = (1/sqrt(2)))
```

Calculates the coefficients

Parameters

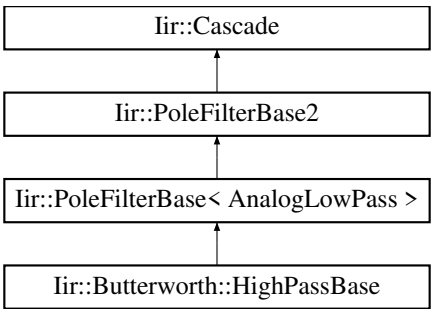
cutoffFrequency	Normalised cutoff frequency (0..1/2)
q	Q factor determines the resonance peak at the cutoff.

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.43 Iir::Butterworth::HighPassBase Struct Reference

Inheritance diagram for Iir::Butterworth::HighPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

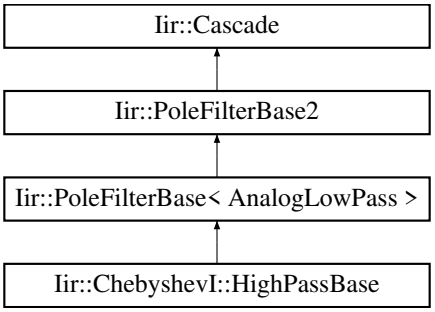
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.44 Iir::ChebyshevI::HighPassBase Struct Reference

Inheritance diagram for Iir::ChebyshevI::HighPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

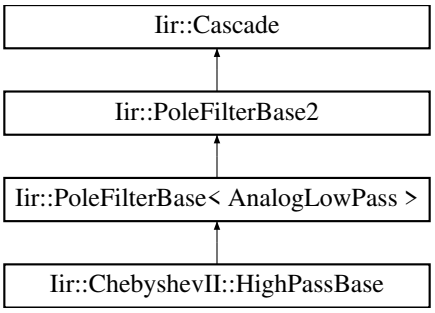
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.45 Iir::ChebyshevII::HighPassBase Struct Reference

Inheritance diagram for Iir::ChebyshevII::HighPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.46 Iir::HighPassTransform Class Reference

`#include <PoleFilter.h>`

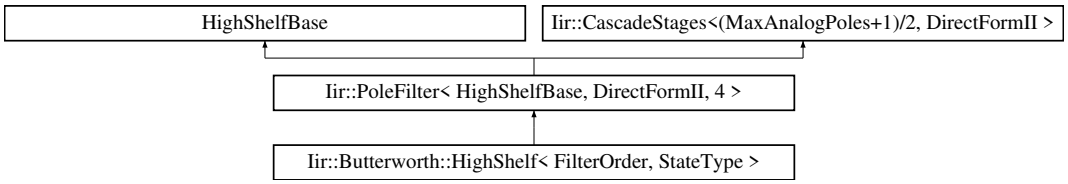
7.46.1 Detailed Description

low pass to high pass
The documentation for this class was generated from the following files:

- iir/PoleFilter.h
- iir/PoleFilter.cpp

7.47 Iir::Butterworth::HighShelf< FilterOrder, StateType > Struct Template Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::HighShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double gainDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double gainDb)
- void setupN (double cutoffFrequency, double gainDb)
- void setupN (int reqOrder, double cutoffFrequency, double gainDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.47.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::Butterworth::HighShelf< FilterOrder, StateType >

Butterworth high shelf filter. Above the cutoff the filter has a specified gain and below it has 0 dB.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.47.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighShelf< FilterOrder, StateType >::setup (
            double sampleRate,
```

```
            double cutoffFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff
gainDb	Gain in dB of the filter in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighShelf< FilterOrder, StateType >::setup (
            int reqOrder,
            double sampleRate,
            double cutoffFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
sampleRate	Sampling rate
cutoffFrequency	Cutoff
gainDb	Gain in dB of the filter in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighShelf< FilterOrder, StateType >::setupN (
            double cutoffFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in dB of the filter in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::HighShelf< FilterOrder, StateType >::setupN (
            int reqOrder,
            double cutoffFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients

Parameters

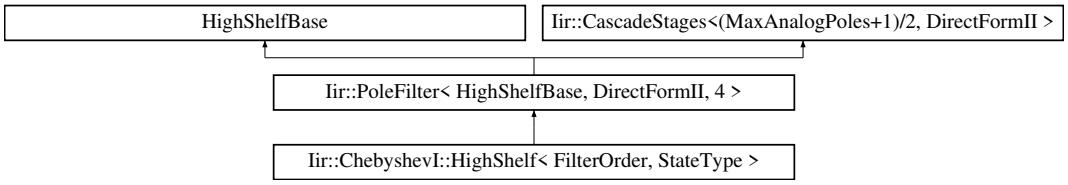
reqOrder	The actual order which can be less than the instantiated one
cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in dB of the filter in the passband

The documentation for this struct was generated from the following file:

- iir/Butterworth.h

7.48 Iir::ChebyshevI::HighShelf< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevI.h>`
Inheritance diagram for Iir::ChebyshevI::HighShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double gainDb, double rippleDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double gainDb, double rippleDb)
- void setupN (double cutoffFrequency, double gainDb, double rippleDb)
- void setupN (int reqOrder, double cutoffFrequency, double gainDb, double rippleDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.48.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevI::HighShelf< FilterOrder, StateType >

ChebyshevI high shelf filter. Specified gain in the passband. Otherwise 0 dB.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.48.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighShelf< FilterOrder, StateType >::setup (
            double sampleRate,
            double cutoffFrequency,
            double gainDb,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighShelf< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double gainDb,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighShelf< FilterOrder, StateType >::setupN (
             double cutoffFrequency,
             double gainDb,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::HighShelf< FilterOrder, StateType >::setupN (
             int reqOrder,
             double cutoffFrequency,
```

```
            double gainDb,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

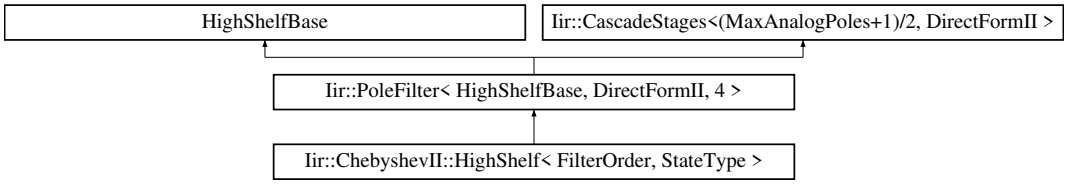
The documentation for this struct was generated from the following file:

- iir/ChebyshevI.h

7.49 Iir::ChebyshevII::HighShelf< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevII.h>`

Inheritance diagram for Iir::ChebyshevII::HighShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double gainDb, double stopBandDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double gainDb, double stopBandDb)
- void setupN (double cutoffFrequency, double gainDb, double stopBandDb)
- void setupN (int reqOrder, double cutoffFrequency, double gainDb, double stopBandDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.49.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevII::HighShelf< FilterOrder, StateType >

ChebyshevII high shelf filter. Specified gain in the passband and 0dB in the stopband.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.49.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighShelf< FilterOrder, StateType >::setup (
             double sampleRate,
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
gainDb	Gain the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighShelf< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
gainDb	Gain the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighShelf< FilterOrder, StateType >::setupN (
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::HighShelf< FilterOrder, StateType >::setupN (
             int reqOrder,
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

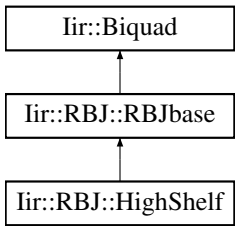
The documentation for this struct was generated from the following file:

- iir/ChebyshevII.h

7.50 Iir::RBJ::HighShelf Struct Reference

`#include <RBJ.h>`

Inheritance diagram for Iir::RBJ::HighShelf:



Public Member Functions

- void setupN (double cutoffFrequency, double gainDb, double shelfSlope=1)
- void setup (double sampleRate, double cutoffFrequency, double gainDb, double shelfSlope=1)

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const

- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.50.1 Detailed Description

High shelf: 0db in the stopband and gainDb in the passband.

7.50.2 Member Function Documentation

setup()

```
void Iir::RBJ::HighShelf::setup (
            double sampleRate,
            double cutoffFrequency,
            double gainDb,
            double shelfSlope = 1)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency
gainDb	Gain in the passband
shelfSlope	Slope between stop/passband. 1 = as steep as it can.

setupN()

```
void Iir::RBJ::HighShelf::setupN (
            double cutoffFrequency,
            double gainDb,
            double shelfSlope = 1)
```

Calculates the coefficients

Parameters

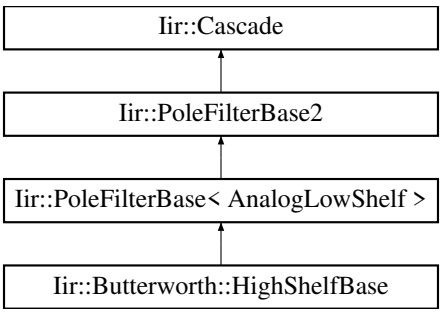
cutoffFrequency	Normalised cutoff frequency
gainDb	Gain in the passband
shelfSlope	Slope between stop/passband. 1 = as steep as it can.

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.51 Iir::Butterworth::HighShelfBase Struct Reference

Inheritance diagram for Iir::Butterworth::HighShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

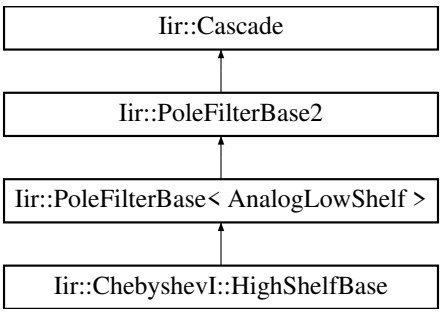
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.52 Iir::ChebyshevI::HighShelfBase Struct Reference

Inheritance diagram for Iir::ChebyshevI::HighShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

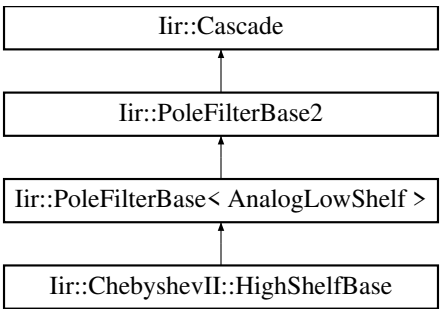
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.53 Iir::ChebyshevII::HighShelfBase Struct Reference

Inheritance diagram for Iir::ChebyshevII::HighShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

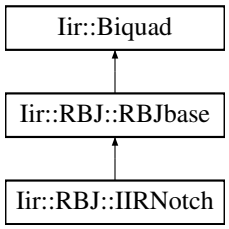
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.54 Iir::RBJ::IIRNotch Struct Reference

`#include <RBJ.h>`
Inheritance diagram for Iir::RBJ::IIRNotch:



Public Member Functions

- void setupN (double centerFrequency, double q_factor=10)
- void setup (double sampleRate, double centerFrequency, double q_factor=10)

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.54.1 Detailed Description

Bandstop with Q factor: the higher the Q factor the more narrow is the notch. However, a narrow notch has a long impulse response (= ringing) and numerical problems might prevent perfect damping. Practical values of the Q factor are about Q = 10 to 20. In terms of the design the Q factor defines the radius of the poles as r = exp(-pi∗(centerFrequency/sampleRate)/q_factor) whereas the angles of the poles/zeros define the bandstop frequency. The higher Q the closer r moves towards the unit circle.

7.54.2 Member Function Documentation

setup()

```
void Iir::RBJ::IIRNotch::setup (
             double sampleRate,
             double centerFrequency,
             double q_factor = 10)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
centerFrequency	Center frequency of the notch
q_factor	Q factor of the notch (1 to ∼20)

setupN()

```
void Iir::RBJ::IIRNotch::setupN (
             double centerFrequency,
             double q_factor = 10)
```

Calculates the coefficients

Parameters

centerFrequency	Normalised centre frequency of the notch
q_factor	Q factor of the notch (1 to ∼20)

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.55 Iir::Layout< MaxPoles > Class Template Reference

`#include <Layout.h>`

7.55.1 Detailed Description

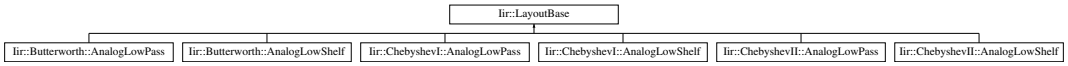
template<int MaxPoles>
class Iir::Layout< MaxPoles >

Storage for Layout
The documentation for this class was generated from the following file:

- iir/Layout.h

7.56 Iir::LayoutBase Class Reference

`#include <Layout.h>`
Inheritance diagram for Iir::LayoutBase:



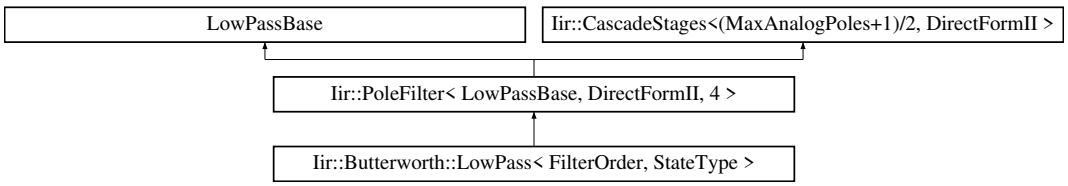
7.56.1 Detailed Description

Base uses pointers to reduce template instantiations
The documentation for this class was generated from the following file:

- iir/Layout.h

7.57 Iir::Butterworth::LowPass< FilterOrder, StateType > Struct Template Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::LowPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency)
- void setupN (double cutoffFrequency)
- void setupN (int reqOrder, double cutoffFrequency)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.57.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::Butterworth::LowPass< FilterOrder, StateType >

Butterworth Lowpass filter.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.57.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowPass< FilterOrder, StateType >::setup (
             double sampleRate,
             double cutoffFrequency)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowPass< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
sampleRate	Sampling rate
cutoffFrequency	Cutoff

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowPass< FilterOrder, StateType >::setupN (
             double cutoffFrequency)  [inline]
```

Calculates the coefficients

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowPass< FilterOrder, StateType >::setupN (
            int reqOrder,
            double cutoffFrequency)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
cutoffFrequency	Normalised cutoff frequency (0..1/2)

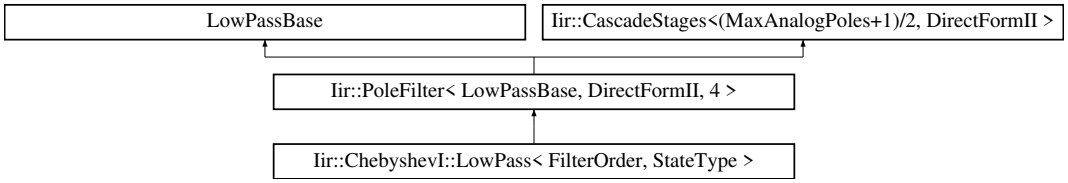
The documentation for this struct was generated from the following file:

- iir/Butterworth.h

7.58 Iir::ChebyshevI::LowPass< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevI.h>`

Inheritance diagram for Iir::ChebyshevI::LowPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double rippleDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double rippleDb)
- void setupN (double cutoffFrequency, double rippleDb)
- void setupN (int reqOrder, double cutoffFrequency, double rippleDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.58.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevI::LowPass< FilterOrder, StateType >

ChebyshevI lowpass filter

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.58.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowPass< FilterOrder, StateType >::setup (
             double sampleRate,
             double cutoffFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency
rippleDb	Permitted ripples in dB in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowPass< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
rippleDb	Permitted ripples in dB in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowPass< FilterOrder, StateType >::setupN (
             double cutoffFrequency,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
rippleDb	Permitted ripples in dB in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowPass< FilterOrder, StateType >::setupN (
             int reqOrder,
```

```
        double cutoffFrequency,
        double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
cutoffFrequency	Normalised cutoff frequency (0..1/2)
rippleDb	Permitted ripples in dB in the passband

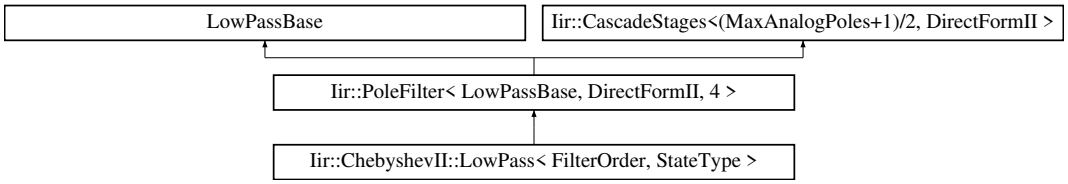
The documentation for this struct was generated from the following file:

- iir/ChebyshevI.h

7.59 Iir::ChebyshevII::LowPass< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevII.h>`

Inheritance diagram for Iir::ChebyshevII::LowPass< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double stopBandDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double stopBandDb)
- void setupN (double cutoffFrequency, double stopBandDb)
- void setupN (int reqOrder, double cutoffFrequency, double stopBandDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.59.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevII::LowPass< FilterOrder, StateType >

ChebyshevII lowpass filter

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.59.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowPass< FilterOrder, StateType >::setup (
            double sampleRate,
            double cutoffFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
stopBandDb	Permitted ripples in dB in the stopband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowPass< FilterOrder, StateType >::setup (
            int reqOrder,
            double sampleRate,
            double cutoffFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowPass< FilterOrder, StateType >::setupN (
            double cutoffFrequency,
            double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
stopBandDb	Permitted ripples in dB in the stopband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowPass< FilterOrder, StateType >::setupN (
            int reqOrder,
```

```
        double cutoffFrequency,
        double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
cutoffFrequency	Normalised cutoff frequency (0..1/2)
stopBandDb	Permitted ripples in dB in the stopband

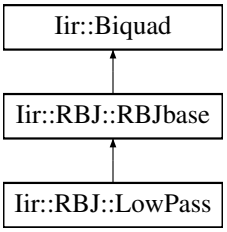
The documentation for this struct was generated from the following file:

- iir/ChebyshevII.h

7.60 Iir::RBJ::LowPass Struct Reference

`#include <RBJ.h>`

Inheritance diagram for Iir::RBJ::LowPass:



Public Member Functions

- void setupN (double cutoffFrequency, double q=(1/sqrt(2)))
- void setup (double sampleRate, double cutoffFrequency, double q=(1/sqrt(2)))

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)

- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.60.1 Detailed Description

Lowpass.

7.60.2 Member Function Documentation

setup()

```
void Iir::RBJ::LowPass::setup (
            double sampleRate,
            double cutoffFrequency,
            double q = (1/sqrt(2)))  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency
q	Q factor determines the resonance peak at the cutoff.

setupN()

```
void Iir::RBJ::LowPass::setupN (
            double cutoffFrequency,
            double q = (1/sqrt(2)))
```

Calculates the coefficients

Parameters

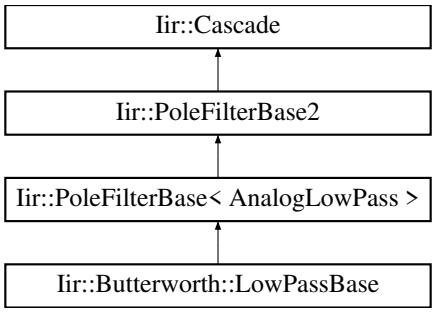
cutoffFrequency	Normalised cutoff frequency
q	Q factor determines the resonance peak at the cutoff.

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.61 Iir::Butterworth::LowPassBase Struct Reference

Inheritance diagram for Iir::Butterworth::LowPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

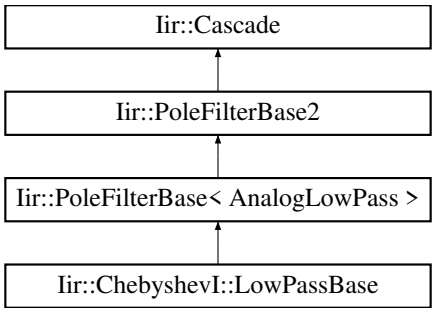
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.62 Iir::ChebyshevI::LowPassBase Struct Reference

Inheritance diagram for Iir::ChebyshevI::LowPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

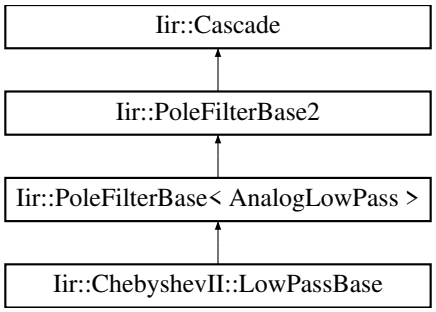
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.63 Iir::ChebyshevII::LowPassBase Struct Reference

Inheritance diagram for Iir::ChebyshevII::LowPassBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.64 Iir::LowPassTransform Class Reference

`#include <PoleFilter.h>`

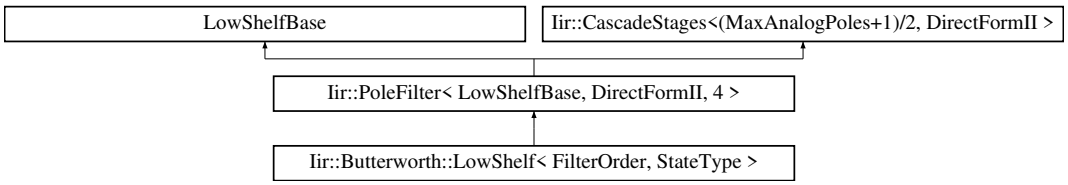
7.64.1 Detailed Description

s-plane to z-plane transforms
For pole filters, an analog prototype is created via placement of poles and zeros in the s-plane. The analog prototype is either a halfband low pass or a halfband low shelf. The poles, zeros, and normalization parameters are transformed into the z-plane using variants of the bilinear transformation. low pass to low pass
The documentation for this class was generated from the following files:

- iir/PoleFilter.h
- iir/PoleFilter.cpp

7.65 Iir::Butterworth::LowShelf< FilterOrder, StateType > Struct Template Reference

`#include <Butterworth.h>`
Inheritance diagram for Iir::Butterworth::LowShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double gainDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double gainDb)
- void setupN (double cutoffFrequency, double gainDb)
- void setupN (int reqOrder, double cutoffFrequency, double gainDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.65.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::Butterworth::LowShelf< FilterOrder, StateType >

Butterworth low shelf filter: below the cutoff it has a specified gain and above the cutoff the gain is 0 dB.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.65.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowShelf< FilterOrder, StateType >::setup (
             double sampleRate,
             double cutoffFrequency,
             double gainDb)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff
gainDb	Gain in dB of the filter in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowShelf< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double gainDb)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
sampleRate	Sampling rate
cutoffFrequency	Cutoff
gainDb	Gain in dB of the filter in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowShelf< FilterOrder, StateType >::setupN (
            double cutoffFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients with the filter order provided by the instantiation

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in dB of the filter in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::Butterworth::LowShelf< FilterOrder, StateType >::setupN (
            int reqOrder,
            double cutoffFrequency,
            double gainDb)  [inline]
```

Calculates the coefficients

Parameters

reqOrder	The actual order which can be less than the instantiated one
cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in dB of the filter in the passband

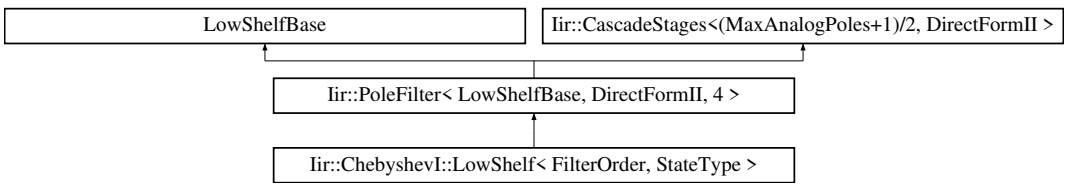
The documentation for this struct was generated from the following file:

- iir/Butterworth.h

7.66 Iir::ChebyshevI::LowShelf< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevI.h>`

Inheritance diagram for Iir::ChebyshevI::LowShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double gainDb, double rippleDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double gainDb, double rippleDb)
- void setupN (double cutoffFrequency, double gainDb, double rippleDb)
- void setupN (int reqOrder, double cutoffFrequency, double gainDb, double rippleDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])

- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.66.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevI::LowShelf< FilterOrder, StateType >

ChebyshevI low shelf filter. Specified gain in the passband. Otherwise 0 dB.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.66.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowShelf< FilterOrder, StateType >::setup (
             double sampleRate,
             double cutoffFrequency,
             double gainDb,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowShelf< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double gainDb,
             double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowShelf< FilterOrder, StateType >::setupN (
            double cutoffFrequency,
            double gainDb,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at the order FilterOrder

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevI::LowShelf< FilterOrder, StateType >::setupN (
            int reqOrder,
            double cutoffFrequency,
            double gainDb,
            double rippleDb)  [inline]
```

Calculates the coefficients of the filter at specified order

Parameters

reqOrder	Actual order for the filter calculations
cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain in the passband
rippleDb	Permitted ripples in dB in the passband

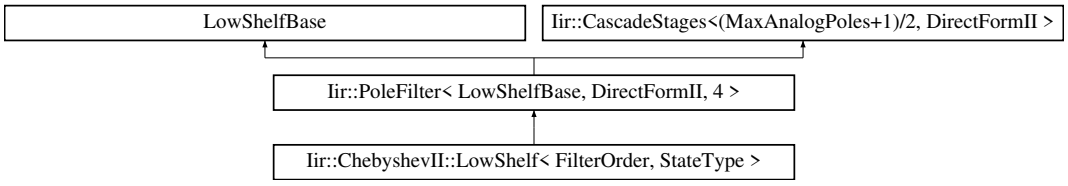
The documentation for this struct was generated from the following file:

- iir/ChebyshevI.h

7.67 Iir::ChebyshevII::LowShelf< FilterOrder, StateType > Struct Template Reference

`#include <ChebyshevII.h>`

Inheritance diagram for Iir::ChebyshevII::LowShelf< FilterOrder, StateType >:



Public Member Functions

- void setup (double sampleRate, double cutoffFrequency, double gainDb, double stopBandDb)
- void setup (int reqOrder, double sampleRate, double cutoffFrequency, double gainDb, double stopBandDb)
- void setupN (double cutoffFrequency, double gainDb, double stopBandDb)
- void setupN (int reqOrder, double cutoffFrequency, double gainDb, double stopBandDb)

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.67.1 Detailed Description

template<int FilterOrder = 4, class StateType = DirectFormII>
struct Iir::ChebyshevII::LowShelf< FilterOrder, StateType >

ChebyshevII low shelf filter. Specified gain in the passband and 0dB in the stopband.

Parameters

FilterOrder	Reserves memory for a filter of the order FilterOrder
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.67.2 Member Function Documentation

setup() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowShelf< FilterOrder, StateType >::setup (
             double sampleRate,
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency.
gainDb	Gain of the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

setup() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowShelf< FilterOrder, StateType >::setup (
             int reqOrder,
             double sampleRate,
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
sampleRate	Sampling rate

cutoffFrequency	Cutoff frequency
gainDb	Gain of the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [1/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowShelf< FilterOrder, StateType >::setupN (
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain of the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

setupN() [2/2]

```
template<int FilterOrder = 4, class StateType = DirectFormII>
void Iir::ChebyshevII::LowShelf< FilterOrder, StateType >::setupN (
             int reqOrder,
             double cutoffFrequency,
             double gainDb,
             double stopBandDb)  [inline]
```

Calculates the coefficients of the filter

Parameters

reqOrder	Requested order which can be less than the instantiated one
cutoffFrequency	Normalised cutoff frequency (0..1/2)
gainDb	Gain the passbard. The stopband has 0 dB gain.
stopBandDb	Permitted ripples in dB in the stopband

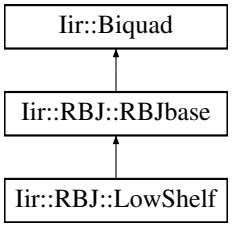
The documentation for this struct was generated from the following file:

- iir/ChebyshevII.h

7.68 Iir::RBJ::LowShelf Struct Reference

`#include <RBJ.h>`

Inheritance diagram for Iir::RBJ::LowShelf:



Public Member Functions

- void setupN (double cutoffFrequency, double gainDb, double shelfSlope=1)
- void setup (double sampleRate, double cutoffFrequency, double gainDb, double shelfSlope=1)

Public Member Functions inherited from Iir::RBJ::RBJbase

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.68.1 Detailed Description

Low shelf: 0db in the stopband and gainDb in the passband.

7.68.2 Member Function Documentation

setup()

```
void Iir::RBJ::LowShelf::setup (
            double sampleRate,
            double cutoffFrequency,
            double gainDb,
            double shelfSlope = 1)  [inline]
```

Calculates the coefficients

Parameters

sampleRate	Sampling rate
cutoffFrequency	Cutoff frequency
gainDb	Gain in the passband
shelfSlope	Slope between stop/passband. 1 = as steep as it can.

setupN()

```
void Iir::RBJ::LowShelf::setupN (
             double cutoffFrequency,
             double gainDb,
             double shelfSlope = 1)
```

Calculates the coefficients

Parameters

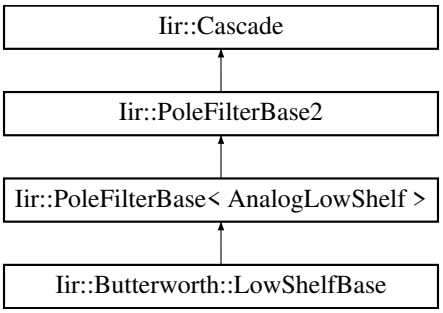
cutoffFrequency	Normalised cutoff frequency
gainDb	Gain in the passband
shelfSlope	Slope between stop/passband. 1 = as steep as it can.

The documentation for this struct was generated from the following files:

- iir/RBJ.h
- iir/RBJ.cpp

7.69 Iir::Butterworth::LowShelfBase Struct Reference

Inheritance diagram for Iir::Butterworth::LowShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

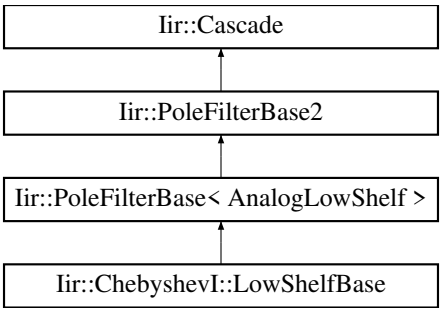
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/Butterworth.h
- iir/Butterworth.cpp

7.70 Iir::ChebyshevI::LowShelfBase Struct Reference

Inheritance diagram for Iir::ChebyshevI::LowShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

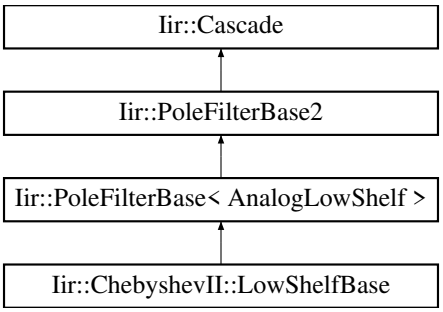
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevI.h
- iir/ChebyshevI.cpp

7.71 Iir::ChebyshevII::LowShelfBase Struct Reference

Inheritance diagram for Iir::ChebyshevII::LowShelfBase:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

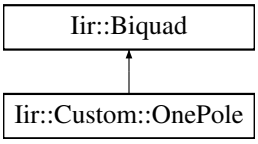
- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

The documentation for this struct was generated from the following files:

- iir/ChebyshevII.h
- iir/ChebyshevII.cpp

7.72 Iir::Custom::OnePole Struct Reference

`#include <Custom.h>`
Inheritance diagram for Iir::Custom::OnePole:



Additional Inherited Members

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.72.1 Detailed Description

Setting up a filter with with one real pole, real zero and scale it by the scale factor

Parameters

scale	Scale the FIR coefficients by this factor
pole	Position of the pole on the real axis
zero	Position of the zero on the real axis

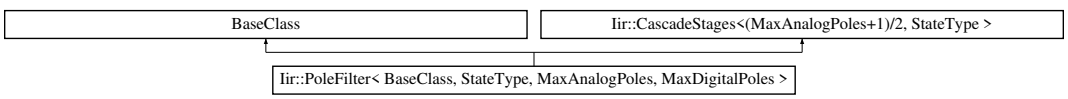
The documentation for this struct was generated from the following files:

- iir/Custom.h
- iir/Custom.cpp

7.73 Iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles > Struct Template Reference

`#include <PoleFilter.h>`

Inheritance diagram for Iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles >:



Additional Inherited Members

Public Member Functions inherited from Iir::CascadeStages<(MaxAnalogPoles+1)/2, StateType >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.73.1 Detailed Description

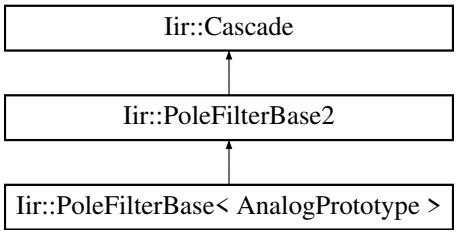
template<class BaseClass, class StateType, int MaxAnalogPoles, int MaxDigitalPoles = MaxAnalogPoles>
struct Iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles >

Storage for pole filters
The documentation for this struct was generated from the following file:

- iir/PoleFilter.h

7.74 Iir::PoleFilterBase< AnalogPrototype > Class Template Reference

`#include <PoleFilter.h>`
Inheritance diagram for Iir::PoleFilterBase< AnalogPrototype >:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

7.74.1 Detailed Description

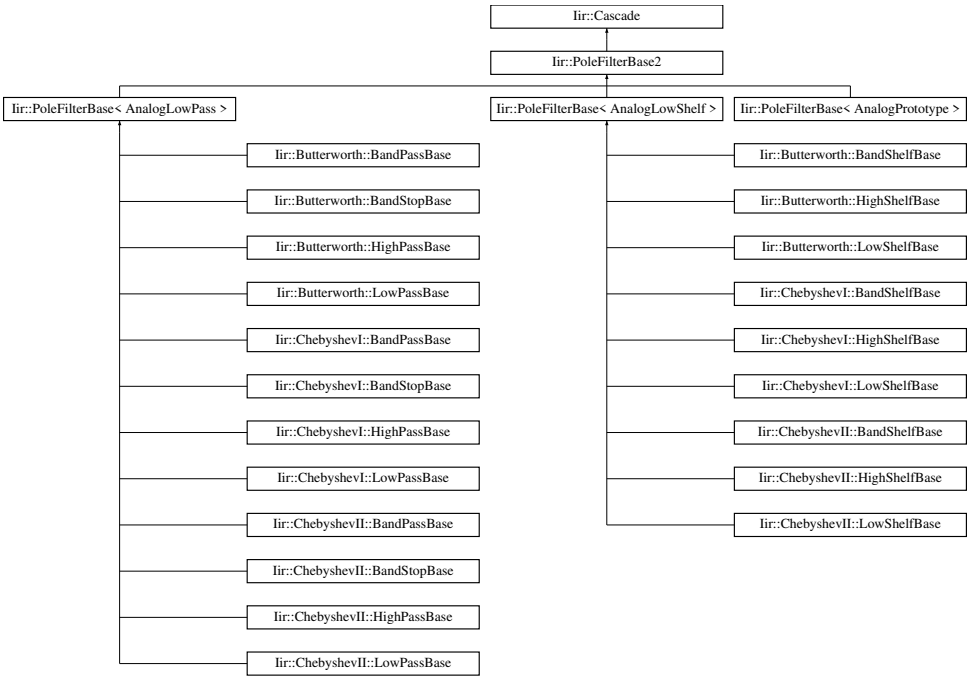
template<class AnalogPrototype>
class Iir::PoleFilterBase< AnalogPrototype >

Serves a container to hold the analog prototype and the digital pole/zero layout.
The documentation for this class was generated from the following file:

- iir/PoleFilter.h

7.75 Iir::PoleFilterBase2 Class Reference

`#include <PoleFilter.h>`
Inheritance diagram for Iir::PoleFilterBase2:



Additional Inherited Members

Public Member Functions inherited from Iir::Cascade

- int getNumStages () const
- const Biquad & operator[ ] (int index)
- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const

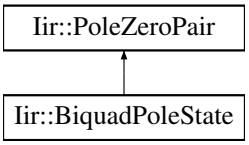
7.75.1 Detailed Description

Factored implementations to reduce template instantiations
The documentation for this class was generated from the following file:

- iir/PoleFilter.h

7.76 Iir::PoleZeroPair Struct Reference

`#include <Types.h>`
Inheritance diagram for Iir::PoleZeroPair:



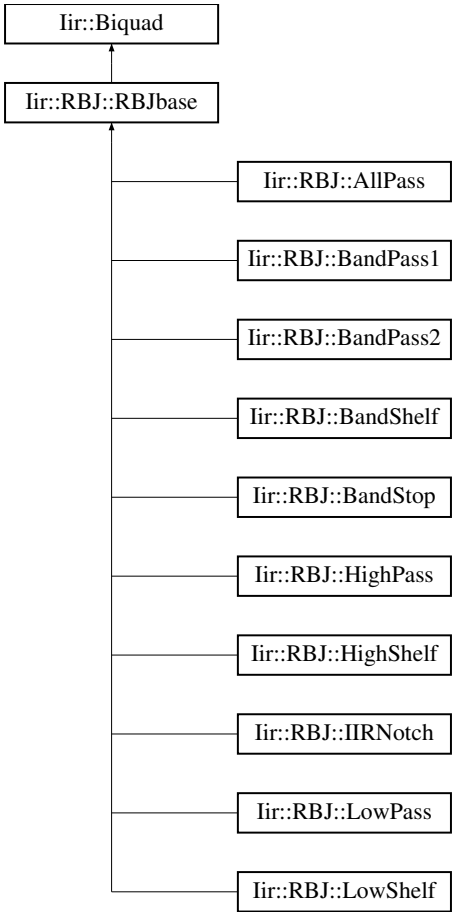
7.76.1 Detailed Description

A pair of pole/zeros. This fits in a biquad (but is missing the gain)
The documentation for this struct was generated from the following file:

- iir/Types.h

7.77 Iir::RBJ::RBJbase Struct Reference

`#include <RBJ.h>`
Inheritance diagram for Iir::RBJ::RBJbase:



Public Member Functions

- template<typename Sample>
  Sample **filter** (Sample s)

  filter operation
- void **reset** ()

  resets the delay lines to zero
- const DirectFormI & **getState** ()

  gets the delay lines (=state) of the filter

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const
- template<class StateType>
  double filter (double s, StateType &state) const

- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

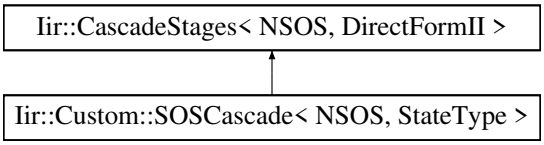
7.77.1 Detailed Description

The base class of all RBJ filters
The documentation for this struct was generated from the following file:

- iir/RBJ.h

7.78 Iir::Custom::SOSCascade< NSOS, StateType > Struct Template Reference

`#include <Custom.h>`
Inheritance diagram for Iir::Custom::SOSCascade< NSOS, StateType >:



Public Member Functions

- SOSCascade ()=default
- SOSCascade (const double(&sosCoefficients)[NSOS][6])
- SOSCascade (const std::array< std::array< double, 6 >, NSOS > sosCoefficients)
- void setup (const double(&sosCoefficients)[NSOS][6])
- void setup (const std::array< std::array< double, 6 >, NSOS > sosCoefficients)

Public Member Functions inherited from Iir::CascadeStages< NSOS, DirectFormII >

- void reset ()
- void setup (const double(&sosCoefficients)[MaxStages][6])
- Sample filter (const Sample in)
- const Cascade::Storage getCascadeStorage ()

7.78.1 Detailed Description

template<std::size_t NSOS, class StateType = DirectFormII>
struct Iir::Custom::SOSCascade< NSOS, StateType >

A custom cascade of 2nd order (SOS / biquads) filters.

Parameters

NSOS	The number of 2nd order filters / biquads.
StateType	The filter topology: DirectFormI, DirectFormII, ...

7.78.2 Constructor & Destructor Documentation

SOSCascade() [1/3]

```
template<std::size_t NSOS, class StateType = DirectFormII>
Iir::Custom::SOSCascade< NSOS, StateType >::SOSCascade ()  [default]
```

Default constructor which creates a unity gain filter of NSOS biquads. Set the filter coefficients later with the setup() method.

SOSCascade() [2/3]

```
template<std::size_t NSOS, class StateType = DirectFormII>
Iir::Custom::SOSCascade< NSOS, StateType >::SOSCascade (
            const double(&) sosCoefficients[NSOS][6])  [inline]
```

Python scipy.signal-friendly setting of coefficients. Initialises the coefficients of the whole chain of biquads / SOS. The argument is a 2D array where the 1st dimension holds an array of 2nd order biquad / SOS coefficients. The six SOS coefficients are ordered "Python" style with first the FIR coefficients (B) and then the IIR coefficients (A). The 2D const double array needs to have exactly the size [NSOS][6].

Parameters

sosCoefficients	2D array Python style sos[NSOS][6]. Indexing: 0-2: FIR-, 3-5: IIR-coefficients.

SOSCascade() [3/3]

```
template<std::size_t NSOS, class StateType = DirectFormII>
Iir::Custom::SOSCascade< NSOS, StateType >::SOSCascade (
            const std::array< std::array< double, 6 >, NSOS > sosCoefficients)  [inline]
```

Python scipy.signal-friendly setting of coefficients. Initialises the coefficients of the whole chain of biquads / SOS. The argument is a 2D array where the 1st dimension holds an array of 2nd order biquad / SOS coefficients. The six SOS coefficients are ordered "Python" style with first the FIR coefficients (B) and then the IIR coefficients (A). The 2D const double array needs to have exactly the size [NSOS][6].

Parameters

sosCoefficients	2D array Python style sos[NSOS][6]. Indexing: 0-2: FIR-, 3-5: IIR-coefficients.

7.78.3 Member Function Documentation

setup() [1/2]

```
template<std::size_t NSOS, class StateType = DirectFormII>
void Iir::Custom::SOSCascade< NSOS, StateType >::setup (
            const double(&) sosCoefficients[NSOS][6])  [inline]
```

Python scipy.signal-friendly setting of coefficients. Sets the coefficients of the whole chain of biquads / SOS. The argument is a 2D array where the 1st dimension holds an array of 2nd order biquad / SOS coefficients. The six SOS coefficients are ordered "Python" style with first the FIR coefficients (B) and then the IIR coefficients (A). The 2D const double array needs to have exactly the size [NSOS][6].

Parameters

sosCoefficients	2D array Python style sos[NSOS][6]. Indexing: 0-2: FIR-, 3-5: IIR-coefficients.

setup() [2/2]

```
template<std::size_t NSOS, class StateType = DirectFormII>
void Iir::Custom::SOSCascade< NSOS, StateType >::setup (
            const std::array< std::array< double, 6 >, NSOS > sosCoefficients)  [inline]
```

Python scipy.signal-friendly setting of coefficients. Sets the coefficients of the whole chain of biquads / SOS. The argument is a 2D array where the 1st dimension holds an array of 2nd order biquad / SOS coefficients. The six SOS coefficients are ordered "Python" style with first the FIR coefficients (B) and then the IIR coefficients (A). The 2D const double array needs to have exactly the size [NSOS][6].

Parameters

sosCoefficients	2D array Python style sos[NSOS][6]. Indexing: 0-2: FIR-, 3-5: IIR-coefficients.

The documentation for this struct was generated from the following file:

- iir/Custom.h

7.79 Iir::Cascade::Storage Struct Reference

`#include <Cascade.h>`

7.79.1 Detailed Description

To return the array from a function and to set it. Transmits number of stages and the pointer to the array.
The documentation for this struct was generated from the following file:

- iir/Cascade.h

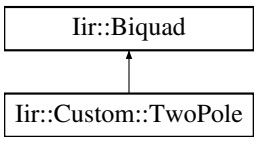
7.80 Iir::TransposedDirectFormII Class Reference

The documentation for this class was generated from the following file:

- iir/State.h

7.81 Iir::Custom::TwoPole Struct Reference

`#include <Custom.h>`
Inheritance diagram for Iir::Custom::TwoPole:



Additional Inherited Members

Public Member Functions inherited from Iir::Biquad

- complex_t response (double normalizedFrequency) const
- std::vector< PoleZeroPair > getPoleZeros () const
- double getA0 () const
- double getA1 () const
- double getA2 () const
- double getB0 () const
- double getB1 () const
- double getB2 () const

- template<class StateType>
  double filter (double s, StateType &state) const
- void setCoefficients (double a0, double a1, double a2, double b0, double b1, double b2)
- void setOnePole (complex_t pole, complex_t zero)
- void setTwoPole (complex_t pole1, complex_t zero1, complex_t pole2, complex_t zero2)
- void setPoleZeroPair (const PoleZeroPair &pair)
- void setIdentity ()
- void applyScale (double scale)

7.81.1 Detailed Description

Set a pole/zero pair in polar coordinates and scale the FIR filter coefficients

Parameters

poleRho	Radius of the pole
poleTheta	Angle of the pole
zeroRho	Radius of the zero
zeroTheta	Angle of the zero

The documentation for this struct was generated from the following files:

- iir/Custom.h
- iir/Custom.cpp

8 File Documentation

8.1 Iir.h

```
00001
00034
00035 #ifndef IIR_H
00036 #define IIR_H
00037
00038 //
00039 // Include this file in your application to get everything
00040 //
00041
00042 #include "iir/Common.h"
00043
00044 #include "iir/Biquad.h"
00045 #include "iir/Cascade.h"
00046 #include "iir/PoleFilter.h"
00047 #include "iir/State.h"
00048
00049 #include "iir/Butterworth.h"
00050 #include "iir/ChebyshevI.h"
00051 #include "iir/ChebyshevII.h"
00052 #include "iir/Custom.h"
00053 #include "iir/RBJ.h"
00054
00055 #endif
```

8.2 Biquad.h

```
00001
00035
00036 #ifndef IIR1_BIQUAD_H
00037 #define IIR1_BIQUAD_H
00038
00039 #include "Common.h"
00040 #include "MathSupplement.h"
00041 #include "Types.h"
00042
00043 namespace Iir {
00044
00045         struct IIR_EXPORT BiquadPoleState;
```

```
00046
00047 /*
00048  * Holds coefficients for a second order Infinite Impulse Response
00049  * digital filter. This is the building block for all IIR filters.
00050  *
00051  */
00052         class IIR_EXPORT Biquad {
00053         public:
00054
00055         Biquad() = default;
00056
00061         complex_t response (double normalizedFrequency) const;
00062
00066         std::vector<PoleZeroPair> getPoleZeros () const;
00067
00071         double getA0 () const { return m_a0; }
00072
00076         double getA1 () const { return m_a1*m_a0; }
00077
00081         double getA2 () const { return m_a2*m_a0; }
00082
00086         double getB0 () const { return m_b0*m_a0; }
00087
00091         double getB1 () const { return m_b1*m_a0; }
00092
00096         double getB2 () const { return m_b2*m_a0; }
00097
00104         template <class StateType>
00105         inline double filter(double s, StateType& state) const
00106         {
00107                 return state.filter(s, *this);
00108         }
00109
00110         public:
00120         void setCoefficients (double a0, double a1, double a2,
00121                               double b0, double b1, double b2);
00122
00126         void setOnePole (complex_t pole, complex_t zero);
00127
00131         void setTwoPole (complex_t pole1, complex_t zero1,
00132                          complex_t pole2, complex_t zero2);
00133
00137         void setPoleZeroPair (const PoleZeroPair& pair)
00138         {
00139                 if (pair.isSinglePole ())
00140                         setOnePole (pair.poles.first, pair.zeros.first);
00141                 else
00142                         setTwoPole (pair.poles.first, pair.zeros.first,
00143                                     pair.poles.second, pair.zeros.second);
00144         }
00145
00146         void setPoleZeroForm (const BiquadPoleState& bps);
00147
00151         void setIdentity ();
00152
00157         void applyScale (double scale);
00158
00159         public:
00160         double m_a0 = 1.0;
00161         double m_a1 = 0.0;
00162         double m_a2 = 0.0;
00163         double m_b1 = 0.0;
00164         double m_b2 = 0.0;
00165         double m_b0 = 1.0;
00166         };
00167
00168 //--------------------------------------------------------------------------
00169
00170
00175         struct IIR_EXPORT BiquadPoleState : PoleZeroPair
00176         {
00177                 BiquadPoleState () = default;
00178
00179                 explicit BiquadPoleState (const Biquad& s);
00180
00181                 double gain = 1.0;
00182         };
00183
00184 }
00185
00186 #endif
```

8.3 Butterworth.h

```
00001
```

```
00035
00036 #ifndef IIR1_BUTTERWORTH_H
00037 #define IIR1_BUTTERWORTH_H
00038
00039 #include "Common.h"
00040 #include "Cascade.h"
00041 #include "PoleFilter.h"
00042 #include "State.h"
00043
00044 namespace Iir {
00045
00052 namespace Butterworth {
00053
00057 class IIR_EXPORT AnalogLowPass : public LayoutBase
00058 {
00059 public:
00060         AnalogLowPass ();
00061
00062         void design (const int numPoles);
00063
00064 private:
00065         int m_numPoles = 0;
00066 };
00067
00068 //------------------------------------------------------------------------------
00069
00073 class IIR_EXPORT AnalogLowShelf : public LayoutBase
00074 {
00075 public:
00076         AnalogLowShelf ();
00077
00078         void design (int numPoles, double gainDb);
00079
00080 private:
00081         int m_numPoles = 0;
00082         double m_gainDb = 0.0;
00083 };
00084
00085 //------------------------------------------------------------------------------
00086
00087 struct IIR_EXPORT LowPassBase : PoleFilterBase <AnalogLowPass>
00088 {
00089         void setup (int order,
00090                     double cutoffFrequency);
00091 };
00092
00093 struct IIR_EXPORT HighPassBase : PoleFilterBase <AnalogLowPass>
00094 {
00095         void setup (int order,
00096                     double cutoffFrequency);
00097 };
00098
00099 struct IIR_EXPORT BandPassBase : PoleFilterBase <AnalogLowPass>
00100 {
00101         void setup (int order,
00102                     double centerFrequency,
00103                     double widthFrequency);
00104 };
00105
00106 struct IIR_EXPORT BandStopBase : PoleFilterBase <AnalogLowPass>
00107 {
00108         void setup (int order,
00109                     double centerFrequency,
00110                     double widthFrequency);
00111 };
00112
00113 struct IIR_EXPORT LowShelfBase : PoleFilterBase <AnalogLowShelf>
00114 {
00115         void setup (int order,
00116                     double cutoffFrequency,
00117                     double gainDb);
00118 };
00119
00120 struct IIR_EXPORT HighShelfBase : PoleFilterBase <AnalogLowShelf>
00121 {
00122         void setup (int order,
00123                     double cutoffFrequency,
00124                     double gainDb);
00125 };
00126
00127 struct IIR_EXPORT BandShelfBase : PoleFilterBase <AnalogLowShelf>
00128 {
00129         void setup (int order,
00130                     double centerFrequency,
00131                     double widthFrequency,
00132                     double gainDb);
00133 };
```

```
00134
00135 //------------------------------------------------------------------------------
00136
00137 //
00138 // Filters for the user
00139 //
00140
00146 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00147 struct LowPass : PoleFilter <LowPassBase, StateType, FilterOrder>
00148 {
00154         void setup (double sampleRate,
00155                     double cutoffFrequency) {
00156                 LowPassBase::setup (FilterOrder,
00157                                     cutoffFrequency / sampleRate);
00158         }
00159
00166         void setup (int reqOrder,
00167                     double sampleRate,
00168                     double cutoffFrequency) {
00169                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00170                 LowPassBase::setup (reqOrder,
00171                                     cutoffFrequency / sampleRate);
00172         }
00173
00174
00179         void setupN(double cutoffFrequency) {
00180                 LowPassBase::setup (FilterOrder,
00181                                     cutoffFrequency);
00182         }
00183
00189         void setupN(int reqOrder,
00190                     double cutoffFrequency) {
00191                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00192                 LowPassBase::setup (reqOrder,
00193                                     cutoffFrequency);
00194         }
00195 };
00196
00202 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00203 struct HighPass : PoleFilter <HighPassBase, StateType, FilterOrder>
00204 {
00205
00211         void setup (double sampleRate,
00212                     double cutoffFrequency) {
00213                 HighPassBase::setup (FilterOrder,
00214                                      cutoffFrequency / sampleRate);
00215         }
00222         void setup (int reqOrder,
00223                     double sampleRate,
00224                     double cutoffFrequency) {
00225                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00226                 HighPassBase::setup (reqOrder,
00227                                      cutoffFrequency / sampleRate);
00228         }
00229
00230
00235         void setupN(double cutoffFrequency) {
00236                 HighPassBase::setup (FilterOrder,
00237                                      cutoffFrequency);
00238         }
00244         void setupN(int reqOrder,
00245                     double cutoffFrequency) {
00246                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00247                 HighPassBase::setup (reqOrder,
00248                                      cutoffFrequency);
00249         }
00250 };
00251
00257 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00258 struct BandPass : PoleFilter <BandPassBase, StateType, FilterOrder, FilterOrder*2>
00259 {
00266         void setup (double sampleRate,
00267                     double centerFrequency,
00268                     double widthFrequency) {
00269                 BandPassBase::setup(FilterOrder,
00270                                     centerFrequency / sampleRate,
00271                                     widthFrequency / sampleRate);
00272         }
00273
00281         void setup (int reqOrder,
00282                     double sampleRate,
00283                     double centerFrequency,
00284                     double widthFrequency) {
00285                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00286                 BandPassBase::setup(reqOrder,
00287                                     centerFrequency / sampleRate,
00288                                     widthFrequency / sampleRate);
```

```
00289         }
00290
00291
00292
00298         void setupN(double centerFrequency,
00299                     double widthFrequency) {
00300                 BandPassBase::setup(FilterOrder,
00301                                     centerFrequency,
00302                                     widthFrequency);
00303         }
00304
00311         void setupN(int reqOrder,
00312                     double centerFrequency,
00313                     double widthFrequency) {
00314                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00315                 BandPassBase::setup(reqOrder,
00316                                     centerFrequency,
00317                                     widthFrequency);
00318         }
00319 };
00320
00321
00327 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00328 struct BandStop : PoleFilter <BandStopBase, StateType, FilterOrder, FilterOrder*2>
00329 {
00336         void setup (double sampleRate,
00337                     double centerFrequency,
00338                     double widthFrequency) {
00339                 BandStopBase::setup (FilterOrder,
00340                                      centerFrequency / sampleRate,
00341                                      widthFrequency / sampleRate);
00342         }
00343
00351         void setup (int reqOrder,
00352                     double sampleRate,
00353                     double centerFrequency,
00354                     double widthFrequency) {
00355                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00356                 BandStopBase::setup (reqOrder,
00357                                      centerFrequency / sampleRate,
00358                                      widthFrequency / sampleRate);
00359         }
00360
00361
00362
00368         void setupN(double centerFrequency,
00369                     double widthFrequency) {
00370                 BandStopBase::setup (FilterOrder,
00371                                      centerFrequency,
00372                                      widthFrequency);
00373         }
00374
00381         void setupN(int reqOrder,
00382                     double centerFrequency,
00383                     double widthFrequency) {
00384                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00385                 BandStopBase::setup (reqOrder,
00386                                      centerFrequency,
00387                                      widthFrequency);
00388         }
00389
00390 };
00391
00398 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00399 struct LowShelf : PoleFilter <LowShelfBase, StateType, FilterOrder>
00400 {
00407         void setup (double sampleRate,
00408                     double cutoffFrequency,
00409                     double gainDb) {
00410                 LowShelfBase::setup (FilterOrder,
00411                                      cutoffFrequency / sampleRate,
00412                                      gainDb);
00413         }
00414
00422         void setup (int reqOrder,
00423                     double sampleRate,
00424                     double cutoffFrequency,
00425                     double gainDb) {
00426                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00427                 LowShelfBase::setup (reqOrder,
00428                                      cutoffFrequency / sampleRate,
00429                                      gainDb);
00430         }
00431
00432
00433
00434
```

```
00440         void setupN(double cutoffFrequency,
00441                     double gainDb) {
00442                 LowShelfBase::setup (FilterOrder,
00443                                      cutoffFrequency,
00444                                      gainDb);
00445         }
00446
00453         void setupN(int reqOrder,
00454                     double cutoffFrequency,
00455                     double gainDb) {
00456                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00457                 LowShelfBase::setup (reqOrder,
00458                                      cutoffFrequency,
00459                                      gainDb);
00460         }
00461
00462 };
00463
00464
00471 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00472 struct HighShelf : PoleFilter <HighShelfBase, StateType, FilterOrder>
00473 {
00480         void setup (double sampleRate,
00481                     double cutoffFrequency,
00482                     double gainDb) {
00483                 HighShelfBase::setup (FilterOrder,
00484                                       cutoffFrequency / sampleRate,
00485                                       gainDb);
00486         }
00487
00495         void setup (int reqOrder,
00496                     double sampleRate,
00497                     double cutoffFrequency,
00498                     double gainDb) {
00499                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00500                 HighShelfBase::setup (reqOrder,
00501                                       cutoffFrequency / sampleRate,
00502                                       gainDb);
00503         }
00504
00505
00506
00512         void setupN(double cutoffFrequency,
00513                     double gainDb) {
00514                 HighShelfBase::setup (FilterOrder,
00515                                       cutoffFrequency,
00516                                       gainDb);
00517         }
00518
00525         void setupN(int reqOrder,
00526                     double cutoffFrequency,
00527                     double gainDb) {
00528                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00529                 HighShelfBase::setup (reqOrder,
00530                                       cutoffFrequency,
00531                                       gainDb);
00532         }
00533 };
00534
00535
00542 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00543 struct BandShelf : PoleFilter <BandShelfBase, StateType, FilterOrder, FilterOrder*2>
00544 {
00552         void setup (double sampleRate,
00553                     double centerFrequency,
00554                     double widthFrequency,
00555                     double gainDb) {
00556                 BandShelfBase::setup (FilterOrder,
00557                                       centerFrequency / sampleRate,
00558                                       widthFrequency / sampleRate,
00559                                       gainDb);
00560         }
00561
00570         void setup (int reqOrder,
00571                     double sampleRate,
00572                     double centerFrequency,
00573                     double widthFrequency,
00574                     double gainDb) {
00575                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00576                 BandShelfBase::setup (reqOrder,
00577                                       centerFrequency / sampleRate,
00578                                       widthFrequency / sampleRate,
00579                                       gainDb);
00580         }
00581
00582
00583
```

```
00590         void setupN(double centerFrequency,
00591                     double widthFrequency,
00592                     double gainDb) {
00593                 BandShelfBase::setup (FilterOrder,
00594                                       centerFrequency,
00595                                       widthFrequency,
00596                                       gainDb);
00597         }
00598
00606         void setupN(int reqOrder,
00607                     double centerFrequency,
00608                     double widthFrequency,
00609                     double gainDb) {
00610                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00611                 BandShelfBase::setup (reqOrder,
00612                                       centerFrequency,
00613                                       widthFrequency,
00614                                       gainDb);
00615         }
00616 };
00617
00618 }
00619
00620 }
00621
00622 #endif
00623
```

8.4 Cascade.h

```
00001
00035
00036 #ifndef IIR1_CASCADE_H
00037 #define IIR1_CASCADE_H
00038
00039 #include "Common.h"
00040 #include "Biquad.h"
00041 #include "Layout.h"
00042 #include "MathSupplement.h"
00043
00044 namespace Iir {
00045
00049         class IIR_EXPORT Cascade
00050         {
00051         public:
00052
00053         Cascade () = default;
00054
00059         struct IIR_EXPORT Storage
00060         {
00061             Storage() = delete;
00062             Storage(const int maxNumBiquads, Biquad* const biquadArray) : maxStages(maxNumBiquads),
      stageArray(biquadArray) {}
00063             const int maxStages = 0;
00064             Biquad* const stageArray = nullptr;
00065         };
00066
00070         int getNumStages () const
00071         {
00072                 return m_numStages;
00073         }
00074
00078         const Biquad& operator[] (int index)
00079         {
00080                 if ((index < 0) || (index >= m_numStages))
00081                         throw_invalid_argument("Index out of bounds.");
00082                 return m_stageArray[index];
00083         }
00084
00089         complex_t response (double normalizedFrequency) const;
00090
00094         std::vector<PoleZeroPair> getPoleZeros () const;
00095
00096         void setCascadeStorage (const Storage& storage);
00097
00098         void applyScale (double scale);
00099
00100         void setLayout (const LayoutBase& proto);
00101
00102         private:
00103         int m_numStages = 0;
00104         int m_maxStages = 0;
00105         Biquad* m_stageArray = nullptr;
00106         };
00107
```

```
00108
00109 //------------------------------------------------------------------------
00110
00115         template <int MaxStages,class StateType>
00116         class CascadeStages {
00117
00118         public:
00119         CascadeStages() = default;
00120
00121
00122         public:
00126         void reset ()
00127         {
00128                 for (auto &state: m_states)
00129                         state.reset();
00130         }
00131
00132         public:
00138         void setup (const double (&sosCoefficients)[MaxStages][6]) {
00139                 for (int i = 0; i < MaxStages; i++) {
00140                         m_stages[i].setCoefficients(
00141                                 sosCoefficients[i][3],
00142                                 sosCoefficients[i][4],
00143                                 sosCoefficients[i][5],
00144                                 sosCoefficients[i][0],
00145                                 sosCoefficients[i][1],
00146                                 sosCoefficients[i][2]);
00147                 }
00148         }
00149
00150         void setup (const std::array<std::array<double,6>,MaxStages> sosCoefficients) {
00151                 for (int i = 0; i < MaxStages; i++) {
00152                         m_stages[i].setCoefficients(
00153                                 sosCoefficients[i][3],
00154                                 sosCoefficients[i][4],
00155                                 sosCoefficients[i][5],
00156                                 sosCoefficients[i][0],
00157                                 sosCoefficients[i][1],
00158                                 sosCoefficients[i][2]);
00159                 }
00160         }
00161
00162         public:
00168         template <typename Sample>
00169         inline Sample filter(const Sample in)
00170         {
00171                 double out = in;
00172                 StateType* state = m_states;
00173                 for (const auto &stage: m_stages)
00174                         out = (state++)->filter(out, stage);
00175                 return static_cast<Sample> (out);
00176         }
00177
00181         const Cascade::Storage getCascadeStorage()
00182         {
00183             const Cascade::Storage s(MaxStages, m_stages);
00184             return s;
00185         }
00186
00187         private:
00188         Biquad m_stages[MaxStages] = {};
00189         StateType m_states[MaxStages] = {};
00190         };
00191
00192 }
00193
00194 #endif
```

8.5 ChebyshevI.h

```
00001
00035
00036 #ifndef IIR1_CHEBYSHEVI_H
00037 #define IIR1_CHEBYSHEVI_H
00038
00039 #include "Common.h"
00040 #include "Cascade.h"
00041 #include "PoleFilter.h"
00042 #include "State.h"
00043
00044 namespace Iir {
00045
00050 namespace ChebyshevI {
00051
00055 class IIR_EXPORT AnalogLowPass : public LayoutBase
```

```
00056 {
00057 public:
00058         AnalogLowPass ();
00059
00060         void design (const int numPoles,
00061                      double rippleDb);
00062
00063 private:
00064         int m_numPoles = 0;
00065         double m_rippleDb = 0.0;
00066 };
00067
00071 class IIR_EXPORT AnalogLowShelf : public LayoutBase
00072 {
00073 public:
00074         AnalogLowShelf ();
00075
00076         void design (int numPoles,
00077                      double gainDb,
00078                      double rippleDb);
00079
00080 private:
00081         int m_numPoles = 0;
00082         double m_rippleDb = 0.0;
00083         double m_gainDb = 0.0;
00084 };
00085
00086 //------------------------------------------------------------------------
00087
00088 struct IIR_EXPORT LowPassBase : PoleFilterBase <AnalogLowPass>
00089 {
00090   void setup (int order,
00091               double cutoffFrequency,
00092               double rippleDb);
00093 };
00094
00095 struct IIR_EXPORT HighPassBase : PoleFilterBase <AnalogLowPass>
00096 {
00097   void setup (int order,
00098               double cutoffFrequency,
00099               double rippleDb);
00100 };
00101
00102 struct IIR_EXPORT BandPassBase : PoleFilterBase <AnalogLowPass>
00103 {
00104   void setup (int order,
00105               double centerFrequency,
00106               double widthFrequency,
00107               double rippleDb);
00108 };
00109
00110 struct IIR_EXPORT BandStopBase : PoleFilterBase <AnalogLowPass>
00111 {
00112   void setup (int order,
00113               double centerFrequency,
00114               double widthFrequency,
00115               double rippleDb);
00116 };
00117
00118 struct IIR_EXPORT LowShelfBase : PoleFilterBase <AnalogLowShelf>
00119 {
00120   void setup (int order,
00121               double cutoffFrequency,
00122               double gainDb,
00123               double rippleDb);
00124 };
00125
00126 struct IIR_EXPORT HighShelfBase : PoleFilterBase <AnalogLowShelf>
00127 {
00128   void setup (int order,
00129               double cutoffFrequency,
00130               double gainDb,
00131               double rippleDb);
00132 };
00133
00134 struct IIR_EXPORT BandShelfBase : PoleFilterBase <AnalogLowShelf>
00135 {
00136   void setup (int order,
00137               double centerFrequency,
00138               double widthFrequency,
00139               double gainDb,
00140               double rippleDb);
00141 };
00142
00143 //------------------------------------------------------------------------
00144
00145 //
```

```
00146 // Userland filters
00147 //
00148
00154 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00155         struct LowPass : PoleFilter <LowPassBase, StateType, FilterOrder>
00156         {
00163                 void setup (double sampleRate,
00164                             double cutoffFrequency,
00165                             double rippleDb) {
00166                         LowPassBase::setup (FilterOrder,
00167                                             cutoffFrequency / sampleRate,
00168                                             rippleDb);
00169                 }
00170
00178                 void setup (int reqOrder,
00179                             double sampleRate,
00180                             double cutoffFrequency,
00181                             double rippleDb) {
00182                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00183                         LowPassBase::setup (reqOrder,
00184                                             cutoffFrequency / sampleRate,
00185                                             rippleDb);
00186                 }
00187
00188
00189
00195                 void setupN(double cutoffFrequency,
00196                             double rippleDb) {
00197                         LowPassBase::setup (FilterOrder,
00198                                             cutoffFrequency,
00199                                             rippleDb);
00200                 }
00201
00208                 void setupN(int reqOrder,
00209                             double cutoffFrequency,
00210                             double rippleDb) {
00211                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00212                         LowPassBase::setup (reqOrder,
00213                                             cutoffFrequency,
00214                                             rippleDb);
00215                 }
00216 };
00217
00223 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00224         struct HighPass : PoleFilter <HighPassBase, StateType, FilterOrder>
00225         {
00232                 void setup (double sampleRate,
00233                             double cutoffFrequency,
00234                             double rippleDb) {
00235                         HighPassBase::setup (FilterOrder,
00236                                              cutoffFrequency / sampleRate,
00237                                              rippleDb);
00238                 }
00239
00247                 void setup (int reqOrder,
00248                             double sampleRate,
00249                             double cutoffFrequency,
00250                             double rippleDb) {
00251                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00252                         HighPassBase::setup (reqOrder,
00253                                              cutoffFrequency / sampleRate,
00254                                              rippleDb);
00255                 }
00256
00257
00258
00264                 void setupN(double cutoffFrequency,
00265                             double rippleDb) {
00266                         HighPassBase::setup (FilterOrder,
00267                                              cutoffFrequency,
00268                                              rippleDb);
00269                 }
00270
00277                 void setupN(int reqOrder,
00278                             double cutoffFrequency,
00279                             double rippleDb) {
00280                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00281                         HighPassBase::setup (reqOrder,
00282                                              cutoffFrequency,
00283                                              rippleDb);
00284                 }
00285 };
00286
00292 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00293         struct BandPass : PoleFilter <BandPassBase, StateType, FilterOrder, FilterOrder*2>
00294         {
00302                 void setup (double sampleRate,
```

```
00303                               double centerFrequency,
00304                               double widthFrequency,
00305                               double rippleDb) {
00306                       BandPassBase::setup (FilterOrder,
00307                               centerFrequency / sampleRate,
00308                               widthFrequency / sampleRate,
00309                               rippleDb);
00310               }
00311
00320               void setup (int reqOrder,
00321                               double sampleRate,
00322                               double centerFrequency,
00323                               double widthFrequency,
00324                               double rippleDb) {
00325                       if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00326                       BandPassBase::setup (reqOrder,
00327                               centerFrequency / sampleRate,
00328                               widthFrequency / sampleRate,
00329                               rippleDb);
00330               }
00331
00332
00333
00340               void setupN(double centerFrequency,
00341                               double widthFrequency,
00342                               double rippleDb) {
00343                       BandPassBase::setup (FilterOrder,
00344                               centerFrequency,
00345                               widthFrequency,
00346                               rippleDb);
00347               }
00348
00356               void setupN(int reqOrder,
00357                               double centerFrequency,
00358                               double widthFrequency,
00359                               double rippleDb) {
00360                       if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00361                       BandPassBase::setup (reqOrder,
00362                               centerFrequency,
00363                               widthFrequency,
00364                               rippleDb);
00365               }
00366 };
00367
00373 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00374         struct BandStop : PoleFilter <BandStopBase, StateType, FilterOrder, FilterOrder*2>
00375         {
00383               void setup (double sampleRate,
00384                               double centerFrequency,
00385                               double widthFrequency,
00386                               double rippleDb) {
00387                       BandStopBase::setup (FilterOrder,
00388                                                   centerFrequency / sampleRate,
00389                                                   widthFrequency / sampleRate,
00390                                                   rippleDb);
00391               }
00392
00401               void setup (int reqOrder,
00402                               double sampleRate,
00403                               double centerFrequency,
00404                               double widthFrequency,
00405                               double rippleDb) {
00406                       if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00407                       BandStopBase::setup (reqOrder,
00408                                                   centerFrequency / sampleRate,
00409                                                   widthFrequency / sampleRate,
00410                                                   rippleDb);
00411               }
00412
00413
00414
00421               void setupN(double centerFrequency,
00422                               double widthFrequency,
00423                               double rippleDb) {
00424                       BandStopBase::setup (FilterOrder,
00425                                                   centerFrequency,
00426                                                   widthFrequency,
00427                                                   rippleDb);
00428               }
00429
00437               void setupN(int reqOrder,
00438                               double centerFrequency,
00439                               double widthFrequency,
00440                               double rippleDb) {
00441                       if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00442                       BandStopBase::setup (reqOrder,
00443                                                   centerFrequency,
```

```
00444                                           widthFrequency,
00445                                           rippleDb);
00446                 }
00447
00448 };
00449
00455 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00456         struct LowShelf : PoleFilter <LowShelfBase, StateType, FilterOrder>
00457         {
00465                 void setup (double sampleRate,
00466                             double cutoffFrequency,
00467                             double gainDb,
00468                             double rippleDb) {
00469                         LowShelfBase::setup (FilterOrder,
00470                                              cutoffFrequency / sampleRate,
00471                                              gainDb,
00472                                              rippleDb);
00473                 }
00474
00483                 void setup (int reqOrder,
00484                             double sampleRate,
00485                             double cutoffFrequency,
00486                             double gainDb,
00487                             double rippleDb) {
00488                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00489                         LowShelfBase::setup (reqOrder,
00490                                              cutoffFrequency / sampleRate,
00491                                              gainDb,
00492                                              rippleDb);
00493                 }
00494
00495
00496
00503                 void setupN(double cutoffFrequency,
00504                             double gainDb,
00505                             double rippleDb) {
00506                         LowShelfBase::setup (FilterOrder,
00507                                              cutoffFrequency,
00508                                              gainDb,
00509                                              rippleDb);
00510                 }
00511
00519                 void setupN(int reqOrder,
00520                             double cutoffFrequency,
00521                             double gainDb,
00522                             double rippleDb) {
00523                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00524                         LowShelfBase::setup (reqOrder,
00525                                              cutoffFrequency,
00526                                              gainDb,
00527                                              rippleDb);
00528                 }
00529 };
00530
00536 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00537         struct HighShelf : PoleFilter <HighShelfBase, StateType, FilterOrder>
00538         {
00546                 void setup (double sampleRate,
00547                             double cutoffFrequency,
00548                             double gainDb,
00549                             double rippleDb) {
00550                         HighShelfBase::setup (FilterOrder,
00551                                               cutoffFrequency / sampleRate,
00552                                               gainDb,
00553                                               rippleDb);
00554                 }
00555
00564                 void setup (int reqOrder,
00565                             double sampleRate,
00566                             double cutoffFrequency,
00567                             double gainDb,
00568                             double rippleDb) {
00569                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00570                         HighShelfBase::setup (reqOrder,
00571                                               cutoffFrequency / sampleRate,
00572                                               gainDb,
00573                                               rippleDb);
00574                 }
00575
00576
00577
00578
00585                 void setupN(double cutoffFrequency,
00586                             double gainDb,
00587                             double rippleDb) {
00588                         HighShelfBase::setup (FilterOrder,
00589                                               cutoffFrequency,
```

```
00590                                   gainDb,
00591                                   rippleDb);
00592                 }
00593
00601                 void setupN(int reqOrder,
00602                             double cutoffFrequency,
00603                             double gainDb,
00604                             double rippleDb) {
00605                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00606                         HighShelfBase::setup (reqOrder,
00607                                 cutoffFrequency,
00608                                 gainDb,
00609                                 rippleDb);
00610                 }
00611
00612 };
00613
00619 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00620         struct BandShelf : PoleFilter <BandShelfBase, StateType, FilterOrder, FilterOrder*2>
00621         {
00630                 void setup (double sampleRate,
00631                             double centerFrequency,
00632                             double widthFrequency,
00633                             double gainDb,
00634                             double rippleDb) {
00635                         BandShelfBase::setup (FilterOrder,
00636                                               centerFrequency / sampleRate,
00637                                               widthFrequency / sampleRate,
00638                                               gainDb,
00639                                               rippleDb);
00640
00641                 }
00642
00652                 void setup (int reqOrder,
00653                             double sampleRate,
00654                             double centerFrequency,
00655                             double widthFrequency,
00656                             double gainDb,
00657                             double rippleDb) {
00658                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00659                         BandShelfBase::setup (reqOrder,
00660                                               centerFrequency / sampleRate,
00661                                               widthFrequency / sampleRate,
00662                                               gainDb,
00663                                               rippleDb);
00664
00665                 }
00666
00667
00668
00669
00677                 void setupN(double centerFrequency,
00678                             double widthFrequency,
00679                             double gainDb,
00680                             double rippleDb) {
00681                         BandShelfBase::setup (FilterOrder,
00682                                               centerFrequency,
00683                                               widthFrequency,
00684                                               gainDb,
00685                                               rippleDb);
00686
00687                 }
00688
00697                 void setupN(int reqOrder,
00698                             double centerFrequency,
00699                             double widthFrequency,
00700                             double gainDb,
00701                             double rippleDb) {
00702                         if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00703                         BandShelfBase::setup (reqOrder,
00704                                               centerFrequency,
00705                                               widthFrequency,
00706                                               gainDb,
00707                                               rippleDb);
00708
00709                 }
00710
00711         };
00712
00713 }
00714
00715 }
00716
00717 #endif
```

8.6 ChebyshevII.h

```
00001
00035
00036 #ifndef IIR1_CHEBYSHEVII_H
00037 #define IIR1_CHEBYSHEVII_H
00038
00039 #include "Common.h"
00040 #include "Cascade.h"
00041 #include "PoleFilter.h"
00042 #include "State.h"
00043
00044 namespace Iir {
00045
00053 namespace ChebyshevII {
00054
00058 class IIR_EXPORT AnalogLowPass : public LayoutBase
00059 {
00060 public:
00061         AnalogLowPass ();
00062
00063         void design (const int numPoles,
00064                      double stopBandDb);
00065
00066 private:
00067         int m_numPoles = 0;
00068         double m_stopBandDb = 0.0;
00069 };
00070
00071
00075 class IIR_EXPORT AnalogLowShelf : public LayoutBase
00076 {
00077 public:
00078         AnalogLowShelf ();
00079
00080         void design (int numPoles,
00081                      double gainDb,
00082                      double stopBandDb);
00083
00084 private:
00085         int m_numPoles = 0;
00086         double m_stopBandDb = 0.0;
00087         double m_gainDb = 0.0;
00088 };
00089
00090 //------------------------------------------------------------------------------
00091
00092 struct IIR_EXPORT LowPassBase : PoleFilterBase <AnalogLowPass>
00093 {
00094         void setup (int order,
00095                     double cutoffFrequency,
00096                     double stopBandDb);
00097 };
00098
00099 struct IIR_EXPORT HighPassBase : PoleFilterBase <AnalogLowPass>
00100 {
00101         void setup (int order,
00102                     double cutoffFrequency,
00103                     double stopBandDb);
00104 };
00105
00106 struct IIR_EXPORT BandPassBase : PoleFilterBase <AnalogLowPass>
00107 {
00108         void setup (int order,
00109                     double centerFrequency,
00110                     double widthFrequency,
00111                     double stopBandDb);
00112 };
00113
00114 struct IIR_EXPORT BandStopBase : PoleFilterBase <AnalogLowPass>
00115 {
00116         void setup (int order,
00117                     double centerFrequency,
00118                     double widthFrequency,
00119                     double stopBandDb);
00120 };
00121
00122 struct IIR_EXPORT LowShelfBase : PoleFilterBase <AnalogLowShelf>
00123 {
00124         void setup (int order,
00125                     double cutoffFrequency,
00126                     double gainDb,
00127                     double stopBandDb);
00128 };
00129
00130 struct IIR_EXPORT HighShelfBase : PoleFilterBase <AnalogLowShelf>
00131 {
```

```
00132         void setup (int order,
00133                     double cutoffFrequency,
00134                     double gainDb,
00135                     double stopBandDb);
00136 };
00137
00138 struct IIR_EXPORT BandShelfBase : PoleFilterBase <AnalogLowShelf>
00139 {
00140         void setup (int order,
00141                     double centerFrequency,
00142                     double widthFrequency,
00143                     double gainDb,
00144                     double stopBandDb);
00145 };
00146
00147 //---------------------------------------------------------------------------
00148
00149 //
00150 // Userland filters
00151 //
00152
00158 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00159 struct LowPass : PoleFilter <LowPassBase, StateType, FilterOrder>
00160 {
00167         void setup (double sampleRate,
00168                     double cutoffFrequency,
00169                     double stopBandDb) {
00170                 LowPassBase::setup (FilterOrder,
00171                                     cutoffFrequency / sampleRate,
00172                                     stopBandDb);
00173         }
00174
00182         void setup (int reqOrder,
00183                     double sampleRate,
00184                     double cutoffFrequency,
00185                     double stopBandDb) {
00186                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00187                 LowPassBase::setup (reqOrder,
00188                                     cutoffFrequency / sampleRate,
00189                                     stopBandDb);
00190         }
00191
00192
00193
00194
00195
00201         void setupN(double cutoffFrequency,
00202                     double stopBandDb) {
00203                 LowPassBase::setup (FilterOrder,
00204                                     cutoffFrequency,
00205                                     stopBandDb);
00206         }
00207
00214         void setupN(int reqOrder,
00215                     double cutoffFrequency,
00216                     double stopBandDb) {
00217                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00218                 LowPassBase::setup (reqOrder,
00219                                     cutoffFrequency,
00220                                     stopBandDb);
00221         }
00222
00223 };
00224
00230 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00231 struct HighPass : PoleFilter <HighPassBase, StateType, FilterOrder>
00232 {
00239         void setup (double sampleRate,
00240                     double cutoffFrequency,
00241                     double stopBandDb) {
00242                 HighPassBase::setup (FilterOrder,
00243                                      cutoffFrequency / sampleRate,
00244                                      stopBandDb);
00245         }
00246
00254         void setup (int reqOrder,
00255                     double sampleRate,
00256                     double cutoffFrequency,
00257                     double stopBandDb) {
00258                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00259                 HighPassBase::setup (reqOrder,
00260                                      cutoffFrequency / sampleRate,
00261                                      stopBandDb);
00262         }
00263
00264
00265
```

```
00266
00272         void setupN(double cutoffFrequency,
00273                     double stopBandDb) {
00274                 HighPassBase::setup (FilterOrder,
00275                                      cutoffFrequency,
00276                                      stopBandDb);
00277         }
00278
00285         void setupN(int reqOrder,
00286                     double cutoffFrequency,
00287                     double stopBandDb) {
00288                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00289                 HighPassBase::setup (reqOrder,
00290                                      cutoffFrequency,
00291                                      stopBandDb);
00292         }
00293
00294 };
00295
00301 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00302 struct BandPass : PoleFilter <BandPassBase, StateType, FilterOrder, FilterOrder*2>
00303 {
00311         void setup (double sampleRate,
00312                     double centerFrequency,
00313                     double widthFrequency,
00314                     double stopBandDb) {
00315                 BandPassBase::setup (FilterOrder,
00316                                      centerFrequency / sampleRate,
00317                                      widthFrequency / sampleRate,
00318                                      stopBandDb);
00319         }
00320
00329         void setup (int reqOrder,
00330                     double sampleRate,
00331                     double centerFrequency,
00332                     double widthFrequency,
00333                     double stopBandDb) {
00334                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00335                 BandPassBase::setup (reqOrder,
00336                                      centerFrequency / sampleRate,
00337                                      widthFrequency / sampleRate,
00338                                      stopBandDb);
00339         }
00340
00341
00342
00343
00350         void setupN(double centerFrequency,
00351                     double widthFrequency,
00352                     double stopBandDb) {
00353                 BandPassBase::setup (FilterOrder,
00354                                      centerFrequency,
00355                                      widthFrequency,
00356                                      stopBandDb);
00357         }
00358
00366         void setupN(int reqOrder,
00367                     double centerFrequency,
00368                     double widthFrequency,
00369                     double stopBandDb) {
00370                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00371                 BandPassBase::setup (reqOrder,
00372                                      centerFrequency,
00373                                      widthFrequency,
00374                                      stopBandDb);
00375         }
00376 };
00377
00383 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00384 struct BandStop : PoleFilter <BandStopBase, StateType, FilterOrder, FilterOrder*2>
00385 {
00393         void setup (double sampleRate,
00394                     double centerFrequency,
00395                     double widthFrequency,
00396                     double stopBandDb) {
00397                 BandStopBase::setup (FilterOrder,
00398                                      centerFrequency / sampleRate,
00399                                      widthFrequency / sampleRate,
00400                                      stopBandDb);
00401         }
00402
00411         void setup (int reqOrder,
00412                     double sampleRate,
00413                     double centerFrequency,
00414                     double widthFrequency,
00415                     double stopBandDb) {
00416                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
```

```
00417                 BandStopBase::setup (reqOrder,
00418                                      centerFrequency / sampleRate,
00419                                      widthFrequency / sampleRate,
00420                                      stopBandDb);
00421         }
00422
00423
00424
00425
00432         void setupN(double centerFrequency,
00433                     double widthFrequency,
00434                     double stopBandDb) {
00435                 BandStopBase::setup (FilterOrder,
00436                                      centerFrequency,
00437                                      widthFrequency,
00438                                      stopBandDb);
00439         }
00440
00448         void setupN(int reqOrder,
00449                     double centerFrequency,
00450                     double widthFrequency,
00451                     double stopBandDb) {
00452                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00453                 BandStopBase::setup (reqOrder,
00454                                      centerFrequency,
00455                                      widthFrequency,
00456                                      stopBandDb);
00457         }
00458 };
00459
00465 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00466 struct LowShelf : PoleFilter <LowShelfBase, StateType, FilterOrder>
00467 {
00475         void setup (double sampleRate,
00476                     double cutoffFrequency,
00477                     double gainDb,
00478                     double stopBandDb) {
00479                 LowShelfBase::setup (FilterOrder,
00480                                      cutoffFrequency / sampleRate,
00481                                      gainDb,
00482                                      stopBandDb);
00483         }
00484
00493         void setup (int reqOrder,
00494                     double sampleRate,
00495                     double cutoffFrequency,
00496                     double gainDb,
00497                     double stopBandDb) {
00498                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00499                 LowShelfBase::setup (reqOrder,
00500                                      cutoffFrequency / sampleRate,
00501                                      gainDb,
00502                                      stopBandDb);
00503         }
00504
00505
00506
00507
00508
00515         void setupN(double cutoffFrequency,
00516                     double gainDb,
00517                     double stopBandDb) {
00518                 LowShelfBase::setup (FilterOrder,
00519                                      cutoffFrequency,
00520                                      gainDb,
00521                                      stopBandDb);
00522         }
00523
00531         void setupN(int reqOrder,
00532                     double cutoffFrequency,
00533                     double gainDb,
00534                     double stopBandDb) {
00535                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00536                 LowShelfBase::setup (reqOrder,
00537                                      cutoffFrequency,
00538                                      gainDb,
00539                                      stopBandDb);
00540         }
00541
00542 };
00543
00549 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00550 struct HighShelf : PoleFilter <HighShelfBase, StateType, FilterOrder>
00551 {
00559         void setup (double sampleRate,
00560                     double cutoffFrequency,
00561                     double gainDb,
```

```
00562                   double stopBandDb) {
00563                 HighShelfBase::setup (FilterOrder,
00564                                       cutoffFrequency / sampleRate,
00565                                       gainDb,
00566                                       stopBandDb);
00567         }
00568
00577         void setup (int reqOrder,
00578                     double sampleRate,
00579                     double cutoffFrequency,
00580                     double gainDb,
00581                     double stopBandDb) {
00582                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00583                 HighShelfBase::setup (reqOrder,
00584                                       cutoffFrequency / sampleRate,
00585                                       gainDb,
00586                                       stopBandDb);
00587         }
00588
00589
00590
00591
00592
00593
00600         void setupN(double cutoffFrequency,
00601                     double gainDb,
00602                     double stopBandDb) {
00603                 HighShelfBase::setup (FilterOrder,
00604                                       cutoffFrequency,
00605                                       gainDb,
00606                                       stopBandDb);
00607         }
00608
00616         void setupN(int reqOrder,
00617                     double cutoffFrequency,
00618                     double gainDb,
00619                     double stopBandDb) {
00620                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00621                 HighShelfBase::setup (reqOrder,
00622                                       cutoffFrequency,
00623                                       gainDb,
00624                                       stopBandDb);
00625         }
00626
00627 };
00628
00634 template <int FilterOrder = DEFAULT_FILTER_ORDER, class StateType = DEFAULT_STATE>
00635 struct BandShelf : PoleFilter <BandShelfBase, StateType, FilterOrder, FilterOrder*2>
00636 {
00645         void setup (double sampleRate,
00646                     double centerFrequency,
00647                     double widthFrequency,
00648                     double gainDb,
00649                     double stopBandDb) {
00650                 BandShelfBase::setup (FilterOrder,
00651                                       centerFrequency / sampleRate,
00652                                       widthFrequency / sampleRate,
00653                                       gainDb,
00654                                       stopBandDb);
00655         }
00656
00657
00667         void setup (int reqOrder,
00668                     double sampleRate,
00669                     double centerFrequency,
00670                     double widthFrequency,
00671                     double gainDb,
00672                     double stopBandDb) {
00673                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00674                 BandShelfBase::setup (reqOrder,
00675                                       centerFrequency / sampleRate,
00676                                       widthFrequency / sampleRate,
00677                                       gainDb,
00678                                       stopBandDb);
00679         }
00680
00681
00682
00683
00684
00685
00686
00694         void setupN(double centerFrequency,
00695                     double widthFrequency,
00696                     double gainDb,
00697                     double stopBandDb) {
00698                 BandShelfBase::setup (FilterOrder,
```

```
00699                                       centerFrequency,
00700                                       widthFrequency,
00701                                       gainDb,
00702                                       stopBandDb);
00703         }
00704
00705
00714         void setupN(int reqOrder,
00715                     double centerFrequency,
00716                     double widthFrequency,
00717                     double gainDb,
00718                     double stopBandDb) {
00719                 if (reqOrder > FilterOrder) throw_invalid_argument(orderTooHigh);
00720                 BandShelfBase::setup (reqOrder,
00721                                       centerFrequency,
00722                                       widthFrequency,
00723                                       gainDb,
00724                                       stopBandDb);
00725         }
00726
00727
00728 };
00729
00730 }
00731
00732 }
00733
00734 #endif
```

8.7 Common.h

```
00001
00035
00036 #ifndef IIR1_COMMON_H
00037 #define IIR1_COMMON_H
00038
00039 //
00040 // This must be the first file included in every DspFilters header and source
00041 //
00042
00043 #ifdef _MSC_VER
00044 #  pragma warning (disable: 4100)
00045 #  ifndef _CRT_SECURE_NO_WARNINGS
00046 #    define _CRT_SECURE_NO_WARNINGS
00047 #  endif
00048 #endif
00049
00050 // This exports the classes/structures to the windows DLL
00051 #if defined(_WIN32) && defined(iir_EXPORTS)
00052 #define IIR_EXPORT __declspec( dllexport )
00053 #else
00054 #define IIR_EXPORT
00055 #endif
00056
00057 #include <stdlib.h>
00058
00059 #include <cassert>
00060 #include <cfloat>
00061 #include <cmath>
00062 #include <complex>
00063 #include <cstring>
00064 #include <string>
00065 #include <limits>
00066 #include <vector>
00067 #include <array>
00068 #include <stdexcept> // for invalid_argument
00069
00070 static const char orderTooHigh[] = "Requested order is too high. Provide a higher order for the
      template.";
00071
00072 #define DEFAULT_FILTER_ORDER 4
00073
00079 inline void throw_invalid_argument(const char* msg) {
00080
00081 #ifndef IIR1_NO_EXCEPTIONS
00082     throw std::invalid_argument(msg);
00083 #else
00084     (void) msg; // Discard parameter
00085     abort();
00086 #endif
00087
00088 }
00089
00090 #endif
```

8.8 Custom.h

```
00001
00035
00036 #ifndef IIR1_CUSTOM_H
00037 #define IIR1_CUSTOM_H
00038
00039 #include "Biquad.h"
00040 #include "Cascade.h"
00041 #include "State.h"
00042
00043 namespace Iir
00044 {
00045
00050
00051 namespace Custom
00052 {
00053
00060 struct OnePole : public Biquad
00061 {
00062     void setup (double scale, double pole, double zero);
00063 };
00064
00072 struct TwoPole : public Biquad
00073 {
00074     void setup (double scale, double poleRho, double poleTheta, double zeroRho,
00075                 double zeroTheta);
00076 };
00077
00083 template <std::size_t NSOS, class StateType = DEFAULT_STATE>
00084 struct SOSCascade : CascadeStages<NSOS, StateType>
00085 {
00090     SOSCascade () = default;
00091
00102     SOSCascade (const double (&sosCoefficients)[NSOS][6])
00103     {
00104         CascadeStages<NSOS, StateType>::setup (sosCoefficients);
00105     }
00106
00117     SOSCascade (const std::array<std::array<double, 6>, NSOS> sosCoefficients)
00118     {
00119         CascadeStages<NSOS, StateType>::setup (sosCoefficients);
00120     }
00121
00132     void setup (const double (&sosCoefficients)[NSOS][6])
00133     {
00134         CascadeStages<NSOS, StateType>::setup (sosCoefficients);
00135     }
00136
00147     void setup (const std::array<std::array<double, 6>, NSOS> sosCoefficients)
00148     {
00149         CascadeStages<NSOS, StateType>::setup (sosCoefficients);
00150     }
00151 };
00152
00153 }
00154
00155 }
00156
00157 #endif
```

8.9 Layout.h

```
00001
00035
00036 #ifndef IIR1_LAYOUT_H
00037 #define IIR1_LAYOUT_H
00038
00039 #include "Common.h"
00040 #include "MathSupplement.h"
00041
00047 namespace Iir {
00048
00049         static const char errPoleisNaN[] = "Pole to add is NaN.";
00050         static const char errZeroisNaN[] = "Zero to add is NaN.";
00051
00052         static const char errCantAdd2ndOrder[] = "Can't add 2nd order after a 1st order filter.";
00053
00054         static const char errPolesNotComplexConj[] = "Poles not complex conjugate.";
00055         static const char errZerosNotComplexConj[] = "Zeros not complex conjugate.";
00056
00057         static const char pairIndexOutOfBounds[] = "Pair index out of bounds.";
00058
00062         class IIR_EXPORT LayoutBase
00063         {
```

```
00064         public:
00065                 LayoutBase ()
00066                         : m_numPoles (0)
00067                         , m_maxPoles (0)
00068                         , m_pair (nullptr)
00069                 {
00070                 }
00071
00072                 LayoutBase (int maxPoles, PoleZeroPair* pairs)
00073                         : m_numPoles (0)
00074                         , m_maxPoles (maxPoles)
00075                         , m_pair (pairs)
00076                 {
00077                 }
00078
00079                 void setStorage (const LayoutBase& other)
00080                 {
00081                         m_numPoles = 0;
00082                         m_maxPoles = other.m_maxPoles;
00083                         m_pair = other.m_pair;
00084                 }
00085
00086                 void reset ()
00087                 {
00088                         m_numPoles = 0;
00089                 }
00090
00091                 int getNumPoles () const
00092                 {
00093                         return m_numPoles;
00094                 }
00095
00096                 int getMaxPoles () const
00097                 {
00098                         return m_maxPoles;
00099                 }
00100
00101                 void add (const complex_t& pole, const complex_t& zero)
00102                 {
00103                         if (m_numPoles&1)
00104                                 throw_invalid_argument(errCantAdd2ndOrder);
00105                         if (Iir::is_nan(pole))
00106                                 throw_invalid_argument(errPoleisNaN);
00107                         if (Iir::is_nan(zero))
00108                                 throw_invalid_argument(errZeroisNaN);
00109                         m_pair[m_numPoles/2] = PoleZeroPair (pole, zero);
00110                         ++m_numPoles;
00111                 }
00112
00113                 void addPoleZeroConjugatePairs (const complex_t& pole,
00114                                                 const complex_t& zero)
00115                 {
00116                         if (m_numPoles&1)
00117                                 throw_invalid_argument(errCantAdd2ndOrder);
00118                         if (Iir::is_nan(pole))
00119                                 throw_invalid_argument(errPoleisNaN);
00120                         if (Iir::is_nan(zero))
00121                                 throw_invalid_argument(errZeroisNaN);
00122                         m_pair[m_numPoles/2] = PoleZeroPair (
00123                                 pole, zero, std::conj (pole), std::conj (zero));
00124                         m_numPoles += 2;
00125                 }
00126
00127                 void add (const ComplexPair& poles, const ComplexPair& zeros)
00128                 {
00129                         if (m_numPoles&1)
00130                                 throw_invalid_argument(errCantAdd2ndOrder);
00131                         if (!poles.isMatchedPair ())
00132                                 throw_invalid_argument(errPolesNotComplexConj);
00133                         if (!zeros.isMatchedPair ())
00134                                 throw_invalid_argument(errZerosNotComplexConj);
00135                         m_pair[m_numPoles/2] = PoleZeroPair (poles.first, zeros.first,
00136                                                              poles.second, zeros.second);
00137                         m_numPoles += 2;
00138                 }
00139
00140                 const PoleZeroPair& getPair (int pairIndex) const
00141                 {
00142                         if ((pairIndex < 0) || (pairIndex >= (m_numPoles+1)/2))
00143                                 throw_invalid_argument(pairIndexOutOfBounds);
00144                         return m_pair[pairIndex];
00145                 }
00146
00147                 const PoleZeroPair& operator[] (int pairIndex) const
00148                 {
00149                         return getPair (pairIndex);
00150                 }
```

```
00151
00152                 double getNormalW () const
00153                 {
00154                         return m_normalW;
00155                 }
00156
00157                 double getNormalGain () const
00158                 {
00159                         return m_normalGain;
00160                 }
00161
00162                 void setNormal (double w, double g)
00163                 {
00164                         m_normalW = w;
00165                         m_normalGain = g;
00166                 }
00167
00168         private:
00169                 int m_numPoles = 0;
00170                 int m_maxPoles = 0;
00171                 PoleZeroPair* m_pair = nullptr;
00172                 double m_normalW = 0;
00173                 double m_normalGain = 1;
00174         };
00175
00176 //------------------------------------------------------------------------
00177
00181         template <int MaxPoles>
00182                 class Layout
00183         {
00184         public:
00185                 operator LayoutBase ()
00186                 {
00187                         return LayoutBase (MaxPoles, m_pairs);
00188                 }
00189
00190         private:
00191                 PoleZeroPair m_pairs[(MaxPoles+1)/2] = {};
00192         };
00193
00194 }
00195
00196 #endif
```

8.10 MathSupplement.h

```
00001
00035
00036 #ifndef IIR1_MATHSUPPLEMENT_H
00037 #define IIR1_MATHSUPPLEMENT_H
00038
00039 #include "Common.h"
00040
00041 #include<complex>
00042
00043 #ifdef _MSC_VER
00044  // Under Unix these have already default instantiations but not under Vis Studio
00045 template class IIR_EXPORT std::complex<double>;
00046 template class IIR_EXPORT std::complex<float>;
00047 #endif
00048
00049 namespace Iir {
00050
00051 const double doublePi   =3.14159265358979323846264338327950288419 71;
00052 const double doublePi_2 =1.57079632679489661923132169163975144209 86;
00053 const double doubleLn2  =0.69314718055994530941723212145818;
00054 const double doubleLn10 =2.30258509299404568401799145468 44;
00055
00056 typedef std::complex<double> complex_t;
00057 typedef std::pair<complex_t, complex_t> complex_pair_t;
00058
00059 inline const complex_t infinity()
00060 {
00061   return complex_t (std::numeric_limits<double>::infinity());
00062 }
00063
00064 template <typename Ty, typename To>
00065 inline std::complex<Ty> addmul (const std::complex<Ty>& c,
00066                                 Ty v,
00067                                 const std::complex<To>& c1)
00068 {
00069   return std::complex <Ty> (
00070     c.real() + v * c1.real(), c.imag() + v * c1.imag());
00071 }
00072
```

```
00073 template <typename Ty>
00074 inline Ty asinh (Ty x)
00075 {
00076   return log (x + std::sqrt (x * x + 1 ));
00077 }
00078
00079 template <typename Ty>
00080 inline bool is_nan (Ty v)
00081 {
00082   return !(v == v);
00083 }
00084
00085 template <>
00086 inline bool is_nan<complex_t> (complex_t v)
00087 {
00088   return Iir::is_nan (v.real()) || Iir::is_nan (v.imag());
00089 }
00090
00091 }
00092
00093 #endif
```

8.11 PoleFilter.h

```
00001
00035
00036 #ifndef IIR1_POLEFILTER_H
00037 #define IIR1_POLEFILTER_H
00038
00039 #include "Common.h"
00040 #include "MathSupplement.h"
00041 #include "Cascade.h"
00042 #include "State.h"
00043
00044 // Purely for debugging...
00045 #include <iostream>
00046
00047 namespace Iir {
00048
00049 /***
00050  * Base for filters designed via algorithmic placement of poles and zeros.
00051  *
00052  * Typically, the filter is first designed as a half-band low pass or
00053  * low shelf analog filter (s-plane). Then, using a transformation such
00054  * as the ones from Constantinides, the poles and zeros of the analog filter
00055  * are calculated in the z-plane.
00056  *
00057  ***/
00058
00062         class IIR_EXPORT PoleFilterBase2 : public Cascade
00063         {
00064         public:
00065                 // This gets the poles/zeros directly from the digital
00066                 // prototype. It is used to double check the correctness
00067                 // of the recovery of pole/zeros from biquad coefficients.
00068                 //
00069                 // It can also be used to accelerate the interpolation
00070                 // of pole/zeros for parameter modulation, since a pole
00071                 // filter already has them calculated
00072
00073                 PoleFilterBase2() = default;
00074
00075                 std::vector<PoleZeroPair> getPoleZeros () const
00076                 {
00077                         std::vector<PoleZeroPair> vpz;
00078                         const int pairs = (m_digitalProto.getNumPoles () + 1) / 2;
00079                         for (int i = 0; i < pairs; ++i)
00080                                 vpz.push_back (m_digitalProto[i]);
00081                         return vpz;
00082                 }
00083
00084         protected:
00085                 LayoutBase m_digitalProto = {};
00086         };
00087
00088
00093         template <class AnalogPrototype>
00094         class PoleFilterBase : public PoleFilterBase2
00095         {
00096         protected:
00097                 void setPrototypeStorage (const LayoutBase& analogStorage,
00098                                           const LayoutBase& digitalStorage)
00099                 {
00100                         m_analogProto.setStorage (analogStorage);
00101                         m_digitalProto = digitalStorage;
```

```
00102                 }
00103
00104         protected:
00105                 AnalogPrototype m_analogProto = {};
00106         };
00107
00108 //------------------------------------------------------------------------
00109
00113         template <class BaseClass,
00114                   class StateType,
00115                   int MaxAnalogPoles,
00116                   int MaxDigitalPoles = MaxAnalogPoles>
00117         struct PoleFilter : BaseClass
00118                 , CascadeStages <(MaxDigitalPoles + 1) / 2 , StateType>
00119         {
00120         public:
00121                 PoleFilter ()
00122                         {
00123                                 // This glues together the factored base classes
00124                                 // with the templatized storage classes.
00125                                 BaseClass::setCascadeStorage (this->getCascadeStorage());
00126                                 BaseClass::setPrototypeStorage (m_analogStorage, m_digitalStorage);
00127                                 CascadeStages<(MaxDigitalPoles + 1) / 2 , StateType>::reset();
00128                         }
00129
00130                 PoleFilter(const PoleFilter&) = default;
00131
00132                 PoleFilter& operator=(const PoleFilter&)
00133                         {
00134                                 // Reset the filter state when copied for now
00135                                 CascadeStages<(MaxDigitalPoles + 1) / 2 , StateType>::reset();
00136                                 return *this;
00137                         }
00138
00139         private:
00140                 Layout <MaxAnalogPoles> m_analogStorage = {};
00141                 Layout <MaxDigitalPoles> m_digitalStorage = {};
00142         };
00143
00144 //------------------------------------------------------------------------
00145
00156
00160         class IIR_EXPORT LowPassTransform
00161         {
00162         public:
00163         LowPassTransform (double fc,
00164                           LayoutBase& digital,
00165                           LayoutBase const& analog);
00166
00167         private:
00168         complex_t transform (complex_t c);
00169
00170         double f = 0.0;
00171         };
00172
00173 //------------------------------------------------------------------------
00174
00178         class IIR_EXPORT HighPassTransform
00179         {
00180         public:
00181         HighPassTransform (double fc,
00182                            LayoutBase& digital,
00183                            LayoutBase const& analog);
00184
00185         private:
00186         complex_t transform (complex_t c);
00187
00188         double f = 0.0;
00189         };
00190
00191 //------------------------------------------------------------------------
00192
00196         class IIR_EXPORT BandPassTransform
00197         {
00198
00199         public:
00200         BandPassTransform (double fc,
00201                            double fw,
00202                            LayoutBase& digital,
00203                            LayoutBase const& analog);
00204
00205         private:
00206         ComplexPair transform (complex_t c);
00207
00208         double wc = 0.0;
00209         double wc2 = 0.0;
00210         double a = 0.0;
```

```
00211         double b = 0.0;
00212         double a2 = 0.0;
00213         double b2 = 0.0;
00214         double ab = 0.0;
00215         double ab_2 = 0.0;
00216         };
00217
00218 //------------------------------------------------------------------------------
00219
00223         class IIR_EXPORT BandStopTransform
00224         {
00225         public:
00226         BandStopTransform (double fc,
00227                            double fw,
00228                            LayoutBase& digital,
00229                            LayoutBase const& analog);
00230
00231         private:
00232         ComplexPair transform (complex_t c);
00233
00234         double wc = 0.0;
00235         double wc2 = 0.0;
00236         double a = 0.0;
00237         double b = 0.0;
00238         double a2 = 0.0;
00239         double b2 = 0.0;
00240         };
00241
00242 }
00243
00244 #endif
```

8.12 RBJ.h

```
00001
00035
00036 #ifndef IIR1_RBJ_H
00037 #define IIR1_RBJ_H
00038
00039 #include "Common.h"
00040 #include "Biquad.h"
00041 #include "State.h"
00042
00043 namespace Iir {
00044
00058
00059 #define ONESQRT2 (1/sqrt(2))
00060
00061 namespace RBJ {
00062
00066         struct IIR_EXPORT RBJbase : Biquad
00067         {
00068         public:
00070                 template <typename Sample>
00071                         inline Sample filter(Sample s) {
00072                         return static_cast<Sample>(state.filter((double)s,*this));
00073                 }
00075                 void reset() {
00076                         state.reset();
00077                 }
00079                 const DirectFormI& getState() {
00080                         return state;
00081                 }
00082         private:
00083                 DirectFormI state = {};
00084         };
00085
00089         struct IIR_EXPORT LowPass : RBJbase
00090         {
00096                 void setupN(double cutoffFrequency,
00097                             double q = ONESQRT2);
00098
00105                 void setup(double sampleRate,
00106                            double cutoffFrequency,
00107                            double q = ONESQRT2) {
00108                         setupN(cutoffFrequency / sampleRate, q);
00109                 }
00110         };
00111
00115         struct IIR_EXPORT HighPass : RBJbase
00116         {
00122                 void setupN(double cutoffFrequency,
00123                             double q = ONESQRT2);
00130                 void setup (double sampleRate,
00131                             double cutoffFrequency,
```

```
00132                           double q = ONESQRT2) {
00133                   setupN(cutoffFrequency / sampleRate, q);
00134           }
00135   };
00136
00140   struct IIR_EXPORT BandPass1 : RBJbase
00141   {
00147           void setupN(double centerFrequency,
00148                       double bandWidth);
00155           void setup (double sampleRate,
00156                       double centerFrequency,
00157                       double bandWidth) {
00158                   setupN(centerFrequency / sampleRate, bandWidth);
00159           }
00160   };
00161
00165   struct IIR_EXPORT BandPass2 : RBJbase
00166   {
00172           void setupN(double centerFrequency,
00173                       double bandWidth);
00180           void setup (double sampleRate,
00181                       double centerFrequency,
00182                       double bandWidth) {
00183                   setupN(centerFrequency / sampleRate, bandWidth);
00184           }
00185   };
00186
00191   struct IIR_EXPORT BandStop : RBJbase
00192   {
00198           void setupN(double centerFrequency,
00199                       double bandWidth);
00206           void setup (double sampleRate,
00207                       double centerFrequency,
00208                       double bandWidth) {
00209                   setupN(centerFrequency / sampleRate, bandWidth);
00210           }
00211   };
00212
00224   struct IIR_EXPORT IIRNotch : RBJbase
00225   {
00231           void setupN(double centerFrequency,
00232                       double q_factor = 10);
00239           void setup (double sampleRate,
00240                       double centerFrequency,
00241                       double q_factor = 10) {
00242                   setupN(centerFrequency / sampleRate, q_factor);
00243           }
00244   };
00245
00249   struct IIR_EXPORT LowShelf : RBJbase
00250   {
00257           void setupN(double cutoffFrequency,
00258                       double gainDb,
00259                       double shelfSlope = 1);
00267           void setup (double sampleRate,
00268                       double cutoffFrequency,
00269                       double gainDb,
00270                       double shelfSlope = 1) {
00271                   setupN( cutoffFrequency / sampleRate, gainDb, shelfSlope);
00272           }
00273   };
00274
00278   struct IIR_EXPORT HighShelf : RBJbase
00279   {
00286           void setupN(double cutoffFrequency,
00287                       double gainDb,
00288                       double shelfSlope = 1);
00296           void setup (double sampleRate,
00297                       double cutoffFrequency,
00298                       double gainDb,
00299                       double shelfSlope = 1) {
00300                   setupN( cutoffFrequency / sampleRate, gainDb, shelfSlope);
00301           }
00302   };
00303
00307   struct IIR_EXPORT BandShelf : RBJbase
00308   {
00315           void setupN(double centerFrequency,
00316                       double gainDb,
00317                       double bandWidth);
00325           void setup (double sampleRate,
00326                       double centerFrequency,
00327                       double gainDb,
00328                       double bandWidth) {
00329                   setupN(centerFrequency / sampleRate, gainDb, bandWidth);
00330           }
00331   };
```

```
00332
00336         struct IIR_EXPORT AllPass : RBJbase
00337         {
00343                 void setupN(double phaseFrequency,
00344                             double q  = ONESQRT2);
00345
00352                 void setup (double sampleRate,
00353                             double phaseFrequency,
00354                             double q  = ONESQRT2) {
00355                     setupN( phaseFrequency / sampleRate, q);
00356                 }
00357         };
00358
00359 }
00360
00361 }
00362
00363
00364 #endif
```

8.13 State.h

```
00001
00035
00036 #ifndef IIR1_STATE_H
00037 #define IIR1_STATE_H
00038
00039 #include "Common.h"
00040 #include "Biquad.h"
00041
00042
00043 #define DEFAULT_STATE DirectFormII
00044
00045 namespace Iir {
00046
00055         class IIR_EXPORT DirectFormI
00056         {
00057         public:
00058         DirectFormI () = default;
00059
00060         void reset ()
00061         {
00062                 m_x1 = 0;
00063                 m_x2 = 0;
00064                 m_y1 = 0;
00065                 m_y2 = 0;
00066         }
00067
00068         inline double filter(const double in,
00069                              const Biquad& s)
00070         {
00071                 const double out = s.m_b0*in + s.m_b1*m_x1 + s.m_b2*m_x2
00072                 - s.m_a1*m_y1 - s.m_a2*m_y2;
00073                 m_x2 = m_x1;
00074                 m_y2 = m_y1;
00075                 m_x1 = in;
00076                 m_y1 = out;
00077
00078                 return out;
00079         }
00080
00081         protected:
00082         double m_x2 = 0.0; // x[n-2]
00083         double m_y2 = 0.0; // y[n-2]
00084         double m_x1 = 0.0; // x[n-1]
00085         double m_y1 = 0.0; // y[n-1]
00086         };
00087
00088 //-----------------------------------------------------------------------------
00089
00099         class IIR_EXPORT DirectFormII
00100         {
00101         public:
00102         DirectFormII () = default;
00103
00104         void reset ()
00105         {
00106                 m_v1 = 0.0;
00107                 m_v2 = 0.0;
00108         }
00109
00110         inline double filter(const double in,
00111                              const Biquad& s)
00112         {
00113                 const double w   = in - s.m_a1*m_v1 - s.m_a2*m_v2;
```

```
00114                 const double out =      s.m_b0*w    + s.m_b1*m_v1 + s.m_b2*m_v2;
00115
00116                 m_v2 = m_v1;
00117                 m_v1 = w;
00118
00119                 return out;
00120         }
00121
00122         private:
00123         double m_v1 = 0.0; // v[-1]
00124         double m_v2 = 0.0; // v[-2]
00125         };
00126
00127
00128 //------------------------------------------------------------------------
00129
00130         class IIR_EXPORT TransposedDirectFormII
00131         {
00132         public:
00133         TransposedDirectFormII() = default;
00134
00135         void reset ()
00136         {
00137                 m_s1 = 0.0;
00138                 m_s1_1 = 0.0;
00139                 m_s2 = 0.0;
00140                 m_s2_1 = 0.0;
00141         }
00142
00143         inline double filter(const double in,
00144                              const Biquad& s)
00145         {
00146                 const double out = m_s1_1 + s.m_b0*in;
00147                 m_s1 = m_s2_1 + s.m_b1*in - s.m_a1*out;
00148                 m_s2 = s.m_b2*in - s.m_a2*out;
00149                 m_s1_1 = m_s1;
00150                 m_s2_1 = m_s2;
00151
00152                 return out;
00153         }
00154
00155         private:
00156         double m_s1 = 0.0;
00157         double m_s1_1 = 0.0;
00158         double m_s2 = 0.0;
00159         double m_s2_1 = 0.0;
00160         };
00161
00162 }
00163
00164 #endif
```

8.14 Types.h

```
00001
00035
00036 #ifndef IIR1_TYPES_H
00037 #define IIR1_TYPES_H
00038
00039 #include "Common.h"
00040 #include "MathSupplement.h"
00041
00042 namespace Iir {
00043
00047         struct IIR_EXPORT ComplexPair : complex_pair_t
00048         {
00049                 ComplexPair() = default;
00050
00051                 explicit ComplexPair (const complex_t& c1)
00052                         : complex_pair_t (c1, 0.)
00053                 {
00054                         if (!isReal()) throw_invalid_argument("A single complex number needs to be
      real.");
00055                 }
00056
00057                 ComplexPair (const complex_t& c1,
00058                              const complex_t& c2)
00059                         : complex_pair_t (c1, c2)
00060                 {
00061                 }
00062
00063                 bool isReal () const
00064                 {
00065                         return first.imag() == 0 && second.imag() == 0;
00066                 }
```

```
00067
00072                 bool isMatchedPair () const
00073                 {
00074                         if (first.imag() != 0)
00075                                 return second == std::conj (first);
00076                         else
00077                                 return second.imag () == 0 &&
00078                                         second.real () != 0 &&
00079                                         first.real () != 0;
00080                 }
00081
00082                 bool is_nan () const
00083                 {
00084                         return Iir::is_nan (first) || Iir::is_nan (second);
00085                 }
00086         };
00087
00088
00092         struct IIR_EXPORT PoleZeroPair
00093         {
00094                 ComplexPair poles = ComplexPair();
00095                 ComplexPair zeros = ComplexPair();
00096
00097                 PoleZeroPair () = default;
00098
00099                 // single pole/zero
00100                 PoleZeroPair (const complex_t& p, const complex_t& z)
00101                         : poles (p), zeros (z)
00102                 {
00103                 }
00104
00105                 // pole/zero pair
00106                 PoleZeroPair (const complex_t& p1, const complex_t& z1,
00107                               const complex_t& p2, const complex_t& z2)
00108                         : poles (p1, p2)
00109                         , zeros (z1, z2)
00110                 {
00111                 }
00112
00113                 bool isSinglePole () const
00114                 {
00115                         return poles.second == 0. && zeros.second == 0.;
00116                 }
00117
00118                 bool is_nan () const
00119                 {
00120                         return poles.is_nan() || zeros.is_nan();
00121                 }
00122         };
00123
00124
00125 }
00126
00127 #endif
```

Index

applyScale
  Iir::Biquad, 47

DSP IIR Realtime C++ filter library, 1

filter
  Iir::Biquad, 47
  Iir::CascadeStages< MaxStages, StateType >, 51

getA0
  Iir::Biquad, 47
getA1
  Iir::Biquad, 47
getA2
  Iir::Biquad, 47
getB0
  Iir::Biquad, 48
getB1
  Iir::Biquad, 48
getB2
  Iir::Biquad, 48
getCascadeStorage
  Iir::CascadeStages< MaxStages, StateType >, 51
getNumStages
  Iir::Cascade, 50
getPoleZeros
  Iir::Biquad, 48
  Iir::Cascade, 50

Iir, 8
iir/Biquad.h, 99
iir/Butterworth.h, 100
iir/Cascade.h, 105
iir/ChebyshevI.h, 106
iir/ChebyshevII.h, 112
iir/Common.h, 117
iir/Custom.h, 118
iir/Layout.h, 118
iir/MathSupplement.h, 120
iir/PoleFilter.h, 121
iir/RBJ.h, 123
iir/State.h, 125
iir/Types.h, 126
Iir::BandPassTransform, 26
Iir::BandStopTransform, 46
Iir::Biquad, 46
  applyScale, 47
  filter, 47
  getA0, 47
  getA1, 47
  getA2, 47
  getB0, 48
  getB1, 48
  getB2, 48
  getPoleZeros, 48
  response, 48
  setCoefficients, 48
  setIdentity, 49
  setOnePole, 49
  setPoleZeroPair, 49
  setTwoPole, 49
Iir::BiquadPoleState, 49
Iir::Butterworth, 11
Iir::Butterworth::AnalogLowPass, 14
Iir::Butterworth::AnalogLowShelf, 15
Iir::Butterworth::BandPass< FilterOrder, StateType >, 16
  setup, 16, 17
  setupN, 17
Iir::Butterworth::BandPassBase, 25
Iir::Butterworth::BandShelf< FilterOrder, StateType >, 27
  setup, 27, 28
  setupN, 28
Iir::Butterworth::BandShelfBase, 35
Iir::Butterworth::BandStop< FilterOrder, StateType >, 37
  setup, 37
  setupN, 38
Iir::Butterworth::BandStopBase, 44
Iir::Butterworth::HighPass< FilterOrder, StateType >, 53
  setup, 54
  setupN, 54
Iir::Butterworth::HighPassBase, 60
Iir::Butterworth::HighShelf< FilterOrder, StateType >, 62
  setup, 62, 63
  setupN, 63
Iir::Butterworth::HighShelfBase, 70
Iir::Butterworth::LowPass< FilterOrder, StateType >, 73
  setup, 74
  setupN, 74, 75
Iir::Butterworth::LowPassBase, 80
Iir::Butterworth::LowShelf< FilterOrder, StateType >, 82
  setup, 83
  setupN, 83, 84
Iir::Butterworth::LowShelfBase, 90
Iir::Cascade, 49
  getNumStages, 50
  getPoleZeros, 50
  operator[], 50
  response, 50
Iir::Cascade::Storage, 98
Iir::CascadeStages< MaxStages, StateType >, 51
  filter, 51
  getCascadeStorage, 51
  reset, 51
  setup, 52
Iir::ChebyshevI, 11
Iir::ChebyshevI::AnalogLowPass, 14

Iir::ChebyshevI::AnalogLowShelf, 15
Iir::ChebyshevI::BandPass< FilterOrder, StateType >, 18
  setup, 18, 19
  setupN, 19
Iir::ChebyshevI::BandPassBase, 25
Iir::ChebyshevI::BandShelf< FilterOrder, StateType >, 29
  setup, 30
  setupN, 30, 31
Iir::ChebyshevI::BandShelfBase, 36
Iir::ChebyshevI::BandStop< FilterOrder, StateType >, 38
  setup, 39
  setupN, 40
Iir::ChebyshevI::BandStopBase, 45
Iir::ChebyshevI::HighPass< FilterOrder, StateType >, 55
  setup, 55, 56
  setupN, 56
Iir::ChebyshevI::HighPassBase, 61
Iir::ChebyshevI::HighShelf< FilterOrder, StateType >, 64
  setup, 64, 65
  setupN, 65
Iir::ChebyshevI::HighShelfBase, 70
Iir::ChebyshevI::LowPass< FilterOrder, StateType >, 75
  setup, 76
  setupN, 76
Iir::ChebyshevI::LowPassBase, 81
Iir::ChebyshevI::LowShelf< FilterOrder, StateType >, 84
  setup, 85
  setupN, 86
Iir::ChebyshevI::LowShelfBase, 90
Iir::ChebyshevII, 12
Iir::ChebyshevII::AnalogLowPass, 14
Iir::ChebyshevII::AnalogLowShelf, 15
Iir::ChebyshevII::BandPass< FilterOrder, StateType >, 20
  setup, 20, 21
  setupN, 21
Iir::ChebyshevII::BandPassBase, 26
Iir::ChebyshevII::BandShelf< FilterOrder, StateType >, 31
  setup, 32
  setupN, 33
Iir::ChebyshevII::BandShelfBase, 36
Iir::ChebyshevII::BandStop< FilterOrder, StateType >, 41
  setup, 41, 42
  setupN, 42
Iir::ChebyshevII::BandStopBase, 45
Iir::ChebyshevII::HighPass< FilterOrder, StateType >, 57
  setup, 57, 58
  setupN, 58
Iir::ChebyshevII::HighPassBase, 61
Iir::ChebyshevII::HighShelf< FilterOrder, StateType >, 66
  setup, 67
  setupN, 67, 68
Iir::ChebyshevII::HighShelfBase, 71
Iir::ChebyshevII::LowPass< FilterOrder, StateType >, 77
  setup, 78
  setupN, 78
Iir::ChebyshevII::LowPassBase, 81
Iir::ChebyshevII::LowShelf< FilterOrder, StateType >, 86
  setup, 87
  setupN, 88
Iir::ChebyshevII::LowShelfBase, 91
Iir::ComplexPair, 52
  isMatchedPair, 52
Iir::Custom, 12
Iir::Custom::OnePole, 91
Iir::Custom::SOSCascade< NSOS, StateType >, 96
  setup, 97
  SOSCascade, 97
Iir::Custom::TwoPole, 98
Iir::DirectFormI, 52
Iir::DirectFormII, 53
Iir::HighPassTransform, 62
Iir::Layout< MaxPoles >, 73
Iir::LayoutBase, 73
Iir::LowPassTransform, 82
Iir::PoleFilter< BaseClass, StateType, MaxAnalogPoles, MaxDigitalPoles >, 92
Iir::PoleFilterBase< AnalogPrototype >, 93
Iir::PoleFilterBase2, 93
Iir::PoleZeroPair, 94
Iir::RBJ::AllPass, 12
  setup, 13
  setupN, 13
Iir::RBJ::BandPass1, 22
  setup, 23
  setupN, 23
Iir::RBJ::BandPass2, 23
  setup, 24
  setupN, 25
Iir::RBJ::BandShelf, 33
  setup, 34
  setupN, 35
Iir::RBJ::BandStop, 43
  setup, 44
  setupN, 44
Iir::RBJ::HighPass, 59
  setup, 60
  setupN, 60
Iir::RBJ::HighShelf, 68
  setup, 69
  setupN, 69
Iir::RBJ::IIRNotch, 71
  setup, 72
  setupN, 72

Iir::RBJ::LowPass, 79
    setup, 80
    setupN, 80
Iir::RBJ::LowShelf, 88
    setup, 89
    setupN, 89
Iir::RBJ::RBJbase, 95
Iir::TransposedDirectFormII, 98
isMatchedPair
    Iir::ComplexPair, 52

operator[]
    Iir::Cascade, 50

reset
    Iir::CascadeStages< MaxStages, StateType >, 51
response
    Iir::Biquad, 48
    Iir::Cascade, 50

setCoefficients
    Iir::Biquad, 48
setIdentity
    Iir::Biquad, 49
setOnePole
    Iir::Biquad, 49
setPoleZeroPair
    Iir::Biquad, 49
setTwoPole
    Iir::Biquad, 49
setup
    Iir::Butterworth::BandPass< FilterOrder, StateType >, 16, 17
    Iir::Butterworth::BandShelf< FilterOrder, StateType >, 27, 28
    Iir::Butterworth::BandStop< FilterOrder, StateType >, 37
    Iir::Butterworth::HighPass< FilterOrder, StateType >, 54
    Iir::Butterworth::HighShelf< FilterOrder, StateType >, 62, 63
    Iir::Butterworth::LowPass< FilterOrder, StateType >, 74
    Iir::Butterworth::LowShelf< FilterOrder, StateType >, 83
    Iir::CascadeStages< MaxStages, StateType >, 52
    Iir::ChebyshevI::BandPass< FilterOrder, StateType >, 18, 19
    Iir::ChebyshevI::BandShelf< FilterOrder, StateType >, 30
    Iir::ChebyshevI::BandStop< FilterOrder, StateType >, 39
    Iir::ChebyshevI::HighPass< FilterOrder, StateType >, 55, 56
    Iir::ChebyshevI::HighShelf< FilterOrder, StateType >, 64, 65
    Iir::ChebyshevI::LowPass< FilterOrder, StateType >, 76
    Iir::ChebyshevI::LowShelf< FilterOrder, StateType >, 85
    Iir::ChebyshevII::BandPass< FilterOrder, StateType >, 20, 21
    Iir::ChebyshevII::BandShelf< FilterOrder, StateType >, 32
    Iir::ChebyshevII::BandStop< FilterOrder, StateType >, 41, 42
    Iir::ChebyshevII::HighPass< FilterOrder, StateType >, 57, 58
    Iir::ChebyshevII::HighShelf< FilterOrder, StateType >, 67
    Iir::ChebyshevII::LowPass< FilterOrder, StateType >, 78
    Iir::ChebyshevII::LowShelf< FilterOrder, StateType >, 87
    Iir::Custom::SOSCascade< NSOS, StateType >, 97
    Iir::RBJ::AllPass, 13
    Iir::RBJ::BandPass1, 23
    Iir::RBJ::BandPass2, 24
    Iir::RBJ::BandShelf, 34
    Iir::RBJ::BandStop, 44
    Iir::RBJ::HighPass, 60
    Iir::RBJ::HighShelf, 69
    Iir::RBJ::IIRNotch, 72
    Iir::RBJ::LowPass, 80
    Iir::RBJ::LowShelf, 89
setupN
    Iir::Butterworth::BandPass< FilterOrder, StateType >, 17
    Iir::Butterworth::BandShelf< FilterOrder, StateType >, 28
    Iir::Butterworth::BandStop< FilterOrder, StateType >, 38
    Iir::Butterworth::HighPass< FilterOrder, StateType >, 54
    Iir::Butterworth::HighShelf< FilterOrder, StateType >, 63
    Iir::Butterworth::LowPass< FilterOrder, StateType >, 74, 75
    Iir::Butterworth::LowShelf< FilterOrder, StateType >, 83, 84
    Iir::ChebyshevI::BandPass< FilterOrder, StateType >, 19
    Iir::ChebyshevI::BandShelf< FilterOrder, StateType >, 30, 31
    Iir::ChebyshevI::BandStop< FilterOrder, StateType >, 40
    Iir::ChebyshevI::HighPass< FilterOrder, StateType >, 56
    Iir::ChebyshevI::HighShelf< FilterOrder, StateType >, 65
    Iir::ChebyshevI::LowPass< FilterOrder, StateType >, 76
    Iir::ChebyshevI::LowShelf< FilterOrder, StateType >, 86
    Iir::ChebyshevII::BandPass< FilterOrder, State-

Type >, 21
Iir::ChebyshevII::BandShelf< FilterOrder, StateType >, 33
Iir::ChebyshevII::BandStop< FilterOrder, StateType >, 42
Iir::ChebyshevII::HighPass< FilterOrder, StateType >, 58
Iir::ChebyshevII::HighShelf< FilterOrder, StateType >, 67, 68
Iir::ChebyshevII::LowPass< FilterOrder, StateType >, 78
Iir::ChebyshevII::LowShelf< FilterOrder, StateType >, 88
Iir::RBJ::AllPass, 13
Iir::RBJ::BandPass1, 23
Iir::RBJ::BandPass2, 25
Iir::RBJ::BandShelf, 35
Iir::RBJ::BandStop, 44
Iir::RBJ::HighPass, 60
Iir::RBJ::HighShelf, 69
Iir::RBJ::IIRNotch, 72
Iir::RBJ::LowPass, 80
Iir::RBJ::LowShelf, 89
SOSCascade
Iir::Custom::SOSCascade< NSOS, StateType >, 97